Kingdom of Saudi Arabia
Ministry of Education
King Faisal University
College of Computer Sciences & Information Technology

Halm: AI-Powered Helmet for Enhanced Worker Protection

A project submitted
In partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science.

by

Hissah Almuhaysh (222433855)

Latifah Alhafith (222431904)

Rana Althafar (222439076)

Ftoon Althafar (222429209)

Supervised by

Dr. Hala Hamdun

Committee Members

Dr. Rawabi Alsedais

March, 2026

ACKNOWLEDGEMENT

Each achievement has a hidden power of faith, the wisdom of mentors, and the unquestioning encouragement of relatives. In all humility, we realize the grace and power of God Almighty, divine guidance, and blessings, which have provided us with both the stamina and nimbleness of intellect to accomplish this strenuous research process, and to him we give this success. Our deepest respect goes to our respected supervisor, Dr. Hala Hamdun, who guided us, believed in us to the highest point, and thought over the perfection of what we did, without whom the course of our direction and the quality of what we are doing would not have happened as it did. We would also like to express our gratitude to Dean and the members of the examination committee who provided their precious criticism and constructive feedback on this work to ensure its academic integrity. It is also due to our families and friends, whose silent sacrifices, patience, and unwavering encouragement were the emotional backbone that helped us through every tough time and doubt; they never doubted us, and so this success belongs to them as much as to us.

UNDERTAKING

This is to declare that the project entitled “Halm: AI-Powered Helmet for Enhanced Worker Protection” is an original work done by the undersigned, in partial fulfillment of the requirements for the degree “Bachelor of Science in Computer Science” at the Department of Computer Science, College of Computer Sciences and Information Technology, King Faisal University.

All the analysis, design, and system development have been accomplished by the undersigned. Moreover, this project has not been submitted to any other college or university.

Hissah Almuhaysh

Latifah Alhafith

Rana Althafar

Ftoon Althafar

ABSTRACT

Table of Contents

List of Tables ix
List of Figures x
1 Introduction 1
1.1 Background and Context 1
1.2 Problem Statement 2
1.3 Objectives 3
1.3.1 Smart Vision Module (O1) 4
1.3.2 Smart Sense System (O2) 4
1.3.3 Centralized Data Analytics Dashboard (O3) 4
1.4 Motivation 4
2 Literature Review 6
2.1 Safety Pillar Analysis of Existing Technologies 6
2.1.1 The Research Network and Scholarly Background Vision 6
2.1.2 Smart Vision Module (O1): From Passive Surveillance to Hazard Localization 7
2.1.3 Smart Sense System (O2): Instantaneous Fall & Impact Detection 8
2.1.4 Centralized Data Analytics Dashboard (O3): Data Fusion & Situational Awareness 8
2.2 Research Gap 9
2.3 Contribution of the Halm Project 9
3 Scope and Deliverables 11
3.1 Definitions of the project's scope 11
3.2 Alignment with KFU identity and societal impact 11
3.3 Expected project deliverables 12
4 Methodology 13
4.1 Requirement Analysis 13
4.2 System design and component architecture 13
4.3 Implementation and Data Pipeline 15
4.4 Detailed Description of the Specific Models 16
4.5 Data Collection and Dataset Preparation 16
4.5.1 Smart Vision (O1) Datasets: Environmental Hazard Localization 17
4.5.2 Smart Sense (O2) Dataset: Fall and Movement Classification 18
4.6 Data Management and Data Security 19

4.7 Testing and Validation 19
4.8 Future Development 20
5 Project Requirements 21
5.1 Functional Requirements 21
5.2 Non-Functional Requirements 21
6 Design and Analysis 23
6.1 System Architecture and Design Justification 23
6.2 System Components and Interaction Flow 24
6.3 System Flow 25
6.3.1 Dual-Path Sensing and Edge Detection 25
6.3.2 Unified Alert and Coordinated Response 26
6.4 Diagrams 27
6.4.1 Database Design Diagram 27
6.4.2 System Flowchart 28
6.4.3 System Architecture Design 29
6.4.4 Hardware Connection Diagram 30
6.4.5 Use Case Diagram 32
7 Technical Development Plan 33
7.1 Technical Choices and Justification of Their Selection 33
7.2 Comparison Of Alternative Solutions 35
7.3 Details Of Tools, Programming Languages, and Platform Used 37
7.3.1 Hardware Components and Embedded Systems 37
7.3.2 Programming Languages and Operating Systems 37
7.3.3 Software and AI Models 38
7.3.4 Design and Diagramming Tools 38
7.3.5 Research Platforms 39
7.3.6 Collaboration Tools 39
7.4 Initial Algorithm (Pseudocode) 40
7.5 Prototype 44
7.5.1 User Authentication and Access Control (4 UIs) 44
7.5.2 Real-Time Dashboard and Situational Awareness (4 UIs) 45
7.5.3 Historical Reports & Analytics (4 UIs) 46
7.5.4 Settings, User Management, and System Configuration (4 UIs) 47

7.6 Ethical & Legal Considerations 48
7.6.1 Privacy and Data Security by Design 48
7.6.2 Autonomy, Transparency, and AI Bias.....48
8 System Implementation and Development.....50
8.1 Implementation Progress Overview 50
8.1.1 Smart Vision Module (O1) – Hazard Detection 50
8.1.2 Smart Sense Module (O2) – Fall Detection.....50
8.1.3 Dashboard and Web System (O3).....51
8.2 Tools and Techniques Utilization 51
8.2.1 Hardware Components.....51
8.2.2 Artificial Intelligence and Computer Vision.....51
8.2.3 Programming Languages 52
8.2.4 Backend and Communication Layer.....52
8.2.5 Frontend and Interface Development.....52
8.3 Design-Implementation Consistency and Optimization 53
8.3.1 System Architecture and Stack Alignment 53
8.3.2 Hardware-Software Integration 53
8.3.3 Unified Multi-Class Detection Pipeline.....54
8.3.4 Dataset Integration and Refinement.....54
8.4 Justification for Design Modification 55
8.4.1 Transition from YOLOv5 to YOLOv8 Small (YOLOv8s) 55
8.4.2 IMU Logic Refinement: From Random Forest to Average Sampling 56
8.4.3 Technical Stack Optimization (React + Vite and Flask-CORS).....56
8.4.4 Database Evolution (Revised ER Diagram and Schema).....57
8.5 Compliance with Professional Practices and Standards.....58
9 System Outcomes and Results.....59
9.1 Technical Model Evaluation and Selection.....59
9.1.1 Visual Evaluation of Model Performance.....60
9.2 Dataset Overview and Experimental Setup.....60
9.2.1 Sharp Tools on the Ground (Class 0).....61
9.2.2 Combining Multiple Pothole / Pit Datasets (Class 1) 61
9.2.3 Handheld Sharp Tools for False Positive Mitigation (Class 2) 61
9.2.4 Experimental Setup and Training Parameters.....62

9.3 Dashboard Interface Implementation 63
9.3.1 User Authentication and Access Control 63
9.3.2 Real-Time Dashboard and Situational Awareness 63
9.3.3 Smart Vision and Live Hazard Localization Implementation 64
9.3.4 Historical Reports and Analytics Implementation 66
9.3.5 Personnel Management and Safety Profiling Implementation 67
9.3.6 Administrative Settings and System Configuration Implementation 67
9.4 Hardware Integration and Edge Processing Implementation 69
9.5 Kinematic Detection and Smart Sense Implementation 70
9.5.1 Installation and Hardware Integration. 70
9.5.2 Fall Detection Algorithm 70
9.5.3 Results and Outcomes: Experiment. 71
10 Challenges and Resolutions 72
10.1 Optimization of AI Model and Data 72
10.1.1 Computation Limitations of Edge Hardware: 72
10.1.2 Fragmentation of Data that does not exist in the real world: 72
10.1.3 Localization inaccuracy in the detection of Sharp Tools: 73
10.1.4 Training Latency and Resource Management: 73
10.2 Hardware Integration/Firmware Problems 73
10.2.1 Critical Hardware Failure (Manufacturing Defect): 73
10.2.2 Power Supply and Amperage Stabilization: 74
10.2.3 CSI Port Compatibility: 74
10.2.4 Firmware and Command Line Driver Errors: 74
10.3 Web Dashboard & Full-Stack Integration 75
10.3.1 UI/UX Design and Elements Hierarchy: 75
10.3.2 Database Schema Evolution for Emergency Response: 75
10.3.3 Transition from Static to Dynamic Data (Flask Integration): 75
11 Work timeline 77
12 Work Timeline for Implementation 78
References 79

List of Tables

Table 1: Literature Review Synthesis Matrix 10
Table 2: Hardware Connection Diagram for the Halm Smart Helmet. 31
Table 3: Comparison of Alternative Solutions. 36
Table 4: Core Hardware Components and Embedded Systems 37
Table 5: Programming Languages and Operating Systems.....37
Table 6: Software Components and Artificial Intelligence Models 38
Table 7: Technical Design and Diagramming Tools.....38
Table 8: Research and Academic Platforms.....39
Table 9: Project Collaboration Platforms 39
Table 10 Hardware Components 51
Table 11 Frontend and Interface Development 52
Table 12: Technical Benchmarking (Nano vs. Small).....59
Table 13: Unified Dataset Composition and Experimental Setup Summary 62

List of Figures

Figure 1: Bibliometric Co-Authorship Network of Smart Safety Research. 7
Figure 2: Multi-Modal Data Preprocessing and Edge Fusion Pipeline 19
Figure 3: Database Design Diagram for the Halm Smart Helmet. 27
Figure 4: System Flowchart for the Halm Smart Helmet. 28
Figure 5: System Architecture Diagram for the Halm Smart Helmet. 29
Figure 6: Hardware Connection Diagram for the Halm Smart Helmet. 30
Figure 7: Use Case Diagram 32
Figure 8: Halm Dashboard User Authentication Interfaces 44
Figure 9: Halm Dashboard Real-Time Situational Awareness Interfaces 45
Figure 10: Halm Dashboard Historical Reports and Analytics Interfaces 46
Figure 11: Halm Dashboard Administrative and Configuration Interfaces 47
Figure 12: Refined Database Schema and Entity Relationships 57
Figure 13: Visual Evaluation of Model Performance 60
Figure 14: Halm Dashboard User Authentication Interfaces 63
Figure 15: Halm Dashboard Real-Time Situational Awareness Interfaces 64
Figure 16: Smart Vision and Live Hazard Localization Implementation 65
Figure 17: Historical Reports and Analytics Implementation 66
Figure 18: Personnel Management and Safety Profiling Implementation 67
Figure 19: Administrative Settings and System Configuration Implementation 68
Figure 20: Administrative Settings and System Configuration Implementation 69
Figure 21: Wiring diagram of the BNO085 IMU sensor connected to the Raspberry Pi GPIO. ... 70
Figure 22: Console output showing successful fall detection and impact force reporting 71
Figure 23: Work Timeline. 77
Figure 24: Work Timeline for Implementation. 78

1 Introduction

The safety of workers in the most physically strenuous industries of the world is one of the challenges of the 21st century. Dangerous workplaces, especially construction and mining operations, are prime sources of accidents, which have led to unacceptable levels of injuries, deaths, and prominent levels of economic losses across the world. These dangers, which range from sudden falls and impact injuries to sharp and localized environmental risks, highlight a warning about the immediate necessity of technological interventions that go beyond conventional and passive protective clothing. In line with the Vision 2030 of Saudi Arabia, which focuses on innovation, sustainable development, and the improvement of the industry and technological capacities of the country, the creation of intelligent safety systems is an essential move towards not only ensuring the safety of workers but also expanding the industrial and technological potential of the country [1].

1.1 Background and Context

The construction and infrastructure industry, although one of the foremost drivers of global and regional economic growth, has the greatest rates of worker injuries and fatalities around the world. This is the tough truth of the imperative of safety innovation. The International Labour Organization (ILO), on a global scale, estimates that hundreds of thousands of workers die every year because of occupational accidents [2], and the construction industry has always recorded the worst rates [3]. As an example, recent data from international safety organizations highlight the global scale of this crisis: The construction sector reported approximately 430,000 accidents worldwide in 2022, tragically resulting in over 6,000 worker fatalities and 330,000 injuries [3]. Furthermore, falls remain the leading cause of these deaths. On a regional scale, in the Kingdom of Saudi Arabia, large-scale infrastructure works are underway at an unprecedented speed as per Vision 2030, which implements world-leading safety standards, leading to an urgent need for smart solutions to protect the workforce and achieve these national safety goals [4]. As illustrated, occupational safety is a challenge in the Kingdom of Saudi Arabia. The symptomatic approach to diagnosing and addressing these accidents proved inefficient in

effectively protecting workers. The latest data issued by the General Organization for Social Insurance (GOSI) shows that the Kingdom witnessed a total of 27,134 occupational injuries in 2023. This mainly resulted from equipment-related accidents (15,352 people) and serious injuries due to fall accidents (7,413 cases) [5]. The accidents are usually caused by avoidable factors that are related to the site conditions and employee welfare, such as falls over the edge, unintentional contact with sharp objects or equipment, and unsafe areas [2]. This is a pressing necessity of strong worker protection, which underlines a significant weakness of the existing safety paradigms: the use of traditional Personal Protective Equipment (PPE), including the conventional hard hat. In principle, such equipment provides passive protection, i.e., it is just post-impact mitigation, which is fundamentally incapable of active/on-the-fly monitoring or advanced and instant risk analysis [6].

The field of Industrial IoT (IIoT), in its attempt to address the shortcomings of passive protection, has produced many wearable concepts. These recent technologies can integrate sensing functions to collect raw data, dealing with single elements of worker risk, including the immediate recognition of gas leakages, sudden falls, or simple physiological strain. The contemporary technological environment, on the other hand, is characterized by functional fragmentation. The solutions are provided as stand-alone modules [7], which are provided in piecemeal format and not as a platform. This drawback does not allow the required integration of external environmental hazards (vision), internal kinetic information (sense), and the fusion of their data into a centralized analysis platform (Dashboard) to build a multi-dimensional comprehensive safety. This underlying absence of the existence of an integrated, holistic system that can provide multi-dimensional safety modeling is what has required the present research to fill a defined and critical gap within next-generation occupational safety technology.

1.2 Problem Statement

The failure to realize the next-generation occupational safety is not that sensing technology is missing, but rather that intelligent integration is not realized. Although past studies have taken substantial steps on the road to making passive Personal Protective Equipment (PPE) a smart wearable gadget, the literature remains highly fragmented and lacks coherent integration across environmental and contextual dimensions [7]. The

solutions are often independent safety modules, which are effective in solving one risk factor and leaving others unchecked. This operational remoteness implies that methods are regularly implemented with emphasis on dangers, such as gauging toxic gases, or fundamental fall alerts, or physiological surveillance [8]. Such a serious restriction does not allow for creating complete safety portraits of a multi-threat worker. Moreover, such intense overreliance on low-value sensor information overlooks the high-value context offered by Computer Vision. As a result, there is an apparent lack of full Smart Vision integration that can autonomously detect, localize, and classify external environmental hazards in real-time (pits, sharp tools). Finally, existing kinematic functionalities, like fall detection, typically rely on basic threshold alerts, resulting in a critical delay that compromises the ability to initiate rapid, real-time intervention during the fall.

Thus, the Halm project would fill the research gap that can be characterized as the deficiency of a single, highly integrated, and anticipatory solution that integrates Smart Vision (environmental hazard detection), Smart Sense (fall detection), and centralized multi-modal Data Analytics into a single, unified smart helmet system. This functional integration ensures the necessary shift from delayed reaction to rapid, coordinated intervention required in time-critical industrial work.

1.3 Objectives

The mandate for safety innovation is to transform the cycle of recording past failures into enabling rapid, informed intervention during critical events by enhancing human capabilities. The proposed Halm project aims to resolve the functional fragmentation observed in existing literature through the creation of a new Halm prototype defined by three highly integrated safety pillars. This system consolidates all critical monitoring functions into a single, integrated platform, ensuring the essential synthesis of external environmental risks to move beyond passive mitigation and enable rapid, integrated safety intelligence. Specifically, the project revolves around the integration of the Smart Vision Module (O1), the Smart Sense System (O2), and a Centralized Data Analytics Dashboard (O3). The following subsections detail the innovation of each pillar.

1.3.1 Smart Vision Module (O1)

Prior research focused on helmet-wearing compliance detection or basic gas sensing, relying on non-visual methods. Halm moves beyond this by pioneering spatial hazard localization (O1) using edge-optimized Computer Vision. This capability grants the worker a digital safety overlay, identifying pits and sharp objects, a core capacity that is absent in earlier smart helmet prototypes, which lacked environmental awareness.

1.3.2 Smart Sense System (O2)

Previous systems were rigid in terms of their primitive detection .The system prototype of Halm is concerned with the Immediate Fall Detection (O2), going beyond the simplified post-incident reporting, which is not enough to intervene in time. This is a strategic decision attributed to the time lag of data processing and sensor input to meet the requirements of instant detection that is reliable. The system will detect on events such as loss of balance and slips during the event, using real-time Inertial Measurement Unit (IMU) data to alert immediately to diminish the consequences of the incident.

1.3.3 Centralized Data Analytics Dashboard (O3)

Previous platforms typically offered fragmented displays of singular data streams (e.g., gas levels on one screen and location on another [4]). Halm's dashboard provides the vital academic and industrial contribution of multi-modal data fusion (O3). It resolves fragmentation by securely integrating all outputs (Vision warnings, Sense detections, and GPS location) onto one interactive map, establishing a unified Situational Awareness Platform for informed, rapid intervention.

1.4 Motivation

Innovation comes not only with modern technology, but also with the ability to save lives. The strategic shift of Halm is driven by a three-part mission comprising humanitarian needs, the National Mission, and the Computer Science Students' Mission. Fueled by the Humanitarian Mission, which seeks to make the construction industry accident rates significantly lower, Halm seeks to make the change in basic assumptions in the idea of safety protection as passive or active, where safety is defined by data and acted upon instead of being passively accepted. This vision is also strategically aligned to the

National Mission since it will enhance the Saudi Vision 2030 by focusing on innovation and sustainable growth and the enhancement of the industrial and technological capabilities of the country [1], which will strengthen King Faisal University's efforts to advance high-quality research that supports the development of a knowledge-based economy [9], [13].

Computer Science students' mission becomes a reality when academic knowledge moves beyond the classroom to present a practical social benefit. This is because, as students of Computer Science, we feel that our aim of learning would be fulfilled if we were able to put together the hardware and software parts, through the acquired knowledge in the university, and help one very essential part of our society, construction workers. Halm confirms the feasibility of a technical platform of multiple sensors that can take advantage of the shortcomings of past piecemeal research. Using the combination of Smart Vision data, Smart Sense kinematics, and the Centralized Data Analytics Dashboard, we prove our professional competencies in practice. This successful integration is not only a technical achievement but also a straight implementation of our promise to apply technology to the idea of human safety and help to improve the well-being of the community.

2 Literature Review

The Halm project proposes the redefinition of safety innovation by transitioning from a reactive approach focused on capturing historical failures to a more active and instantaneous detection model, ensuring rapid response to imminent threats with advanced sensor data and human-machine interfaces. Though previous literature provides the basis for wearable-based safety equipment and IoT-enabled smart helmets, it is limited to functional fragmentation and is based on reactive mechanisms. The review below synthesizes previous studies across the three pillars of safety as they relate to the Halm project.

2.1 Safety Pillar Analysis of Existing Technologies

The following discussion is a critical review of the current state of the art in wearable occupational safety systems as well as a classification of current research based on the three technical pillars of the Halm project. The analysis is put into the context of Smart Vision (2.1.2), Smart Sense (2.1.3), and Centralized Data Analytics (2.1.4). With the synthesis of such literature, the review presents systemic constraints of existing detection systems and thus defines an area of knowledge gap that the Halm project will seek to fill.

2.1.1 The Research Network and Scholarly Background Vision

Figure 1 presents a bibliometric co-authorship network generated through RABBITResearch, illustrating the interconnections among influential publications and authors relevant to the project's research domain. The network visually maps the intellectual structure of the field, highlighting major research clusters and their temporal evolution. The central node, Jeong et al. [10], appears as a pivotal early contributor, forming the foundation for multiple subsequent studies such as Lee [14], Choi [15], and Kumar [16]. This cluster represents the progression from fundamental theoretical contributions to more recent applied and data-driven approaches.

Additionally, the branch centered on Navasree [17], and Suresh [18] signifies a parallel research stream that extends the earlier methodologies toward enhanced contextual or domain-specific implementations. The dense connectivity among authors such as

Battistoni [19], Roy [20], and Kalita [21] indicates active collaboration and sustained exploration of related subtopics.

Overall, the RABBITResearch visualization confirms that the area under study is characterized by cumulative development and scholarly continuity, where earlier foundational works have inspired newer, more specialized research directions. This bibliometric insight supports the identification of key contributors, recurring themes, and research gaps that the current project aims to address.

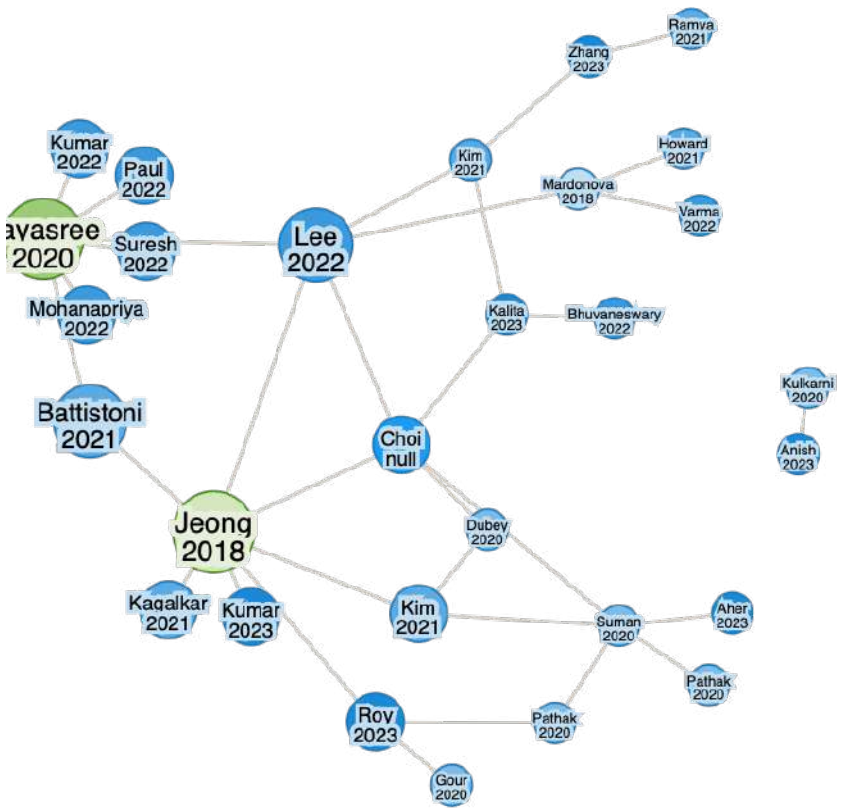

Figure 1: Bibliometric Co-Authorship Network of Smart Safety Research.

2.1.2 Smart Vision Module (O1): From Passive Surveillance to Hazard Localization

In modern visual safety systems, cameras are mainly employed in surveillance and live-casting systems so that they can monitor people who are far away. As an example, a disaster-response helmet has been proposed, combining a thermal and visible camera with aerial views to offer situational awareness [10]. Likewise, helmets that are IoT enabled

have been applied to track the construction process [8], [2]. Recent studies have also demonstrated the use of advanced vision techniques primarily for helmet-wearing compliance detection [14], [15], [16]. Nevertheless, such systems do not include autonomous processing of the environment but still depend on human interpretation. Recent research in autonomous systems shows that edge-optimal Computer Vision (CV) can be put into practice to localize hazards. As an illustration, a Vision-Language Model (VLM) called INSIGHT was proposed to forecast hazards in a driving scenario according to the circumstance [11]. Although not applied to wearables, this paper shows that CV can be used to offer spatial hazard localization, which outperforms passive surveillance. The existing helmet platforms have not included such AI-based environmental awareness.

2.1.3 Smart Sense System (O2): Instantaneous Fall & Impact Detection

Acceleration sensors and gyroscopes are commonly used by wearable safety gadgets to detect falls. Systems with a threshold give an alarm when an impact has already taken place [3], [4], [8]. Although these mechanisms have effectively been applied to incident reporting, they can only be used to detect incidents after the impact has been made, creating a critical delay. Recent advances are more inclined to instantaneous detection [17], [18] to intervene in time. While previous research experimented with complex predictive models (e.g., RRNs in SAFE-Net [7]), the current technological mandate is for reliable, instantaneous detection of falls and impacts, rather than prediction. The system will rely on real-time Inertial Measurement Unit (IMU) data to alert immediately, diminishing the consequences of the incident.

2.1.4 Centralized Data Analytics Dashboard (O3): Data Fusion & Situational Awareness

Most smart helmet systems have central monitoring dashboards, frequently based on an IoT cloud platform such as ThingSpeak [2]. However, these are typically not built-in, presenting discontinuous data streams (e.g., gas, vitals, or position). The importance of integrated multimedia platforms to include multiple inputs to make such decisions is frequently stressed [3], [12]. Nevertheless, despite this appreciation, the solutions that are in place are fragmented. No published prototype exists in which multi-modal integration

of data between vision and motion is integrated into a single instantaneous detection and decision support product [20], [21], [22].

2.2 Research Gap

Despite the clearly identifiable incremental change throughout the three pillars, the literature indicates a critical set of limitations associated with integrated, real-time safety systems. Environmental surveillance is consistently confined to passive surveillance rather than autonomous hazard localizations; kinematic surveillance is limited to reactive, post-facto detection of impacts; and data platforms are functionally decomposed, showing individual parameters rather than fused multi-modal insights. The primary challenge is not sensor technology but the lack of a unified detection architecture that ensures instantaneous, high-fidelity alerts across all dimensions simultaneously. Ultimately, no reported prototype fully combines multi-modal sensing with holistic, instantaneous intelligence.

2.3 Contribution of the Halm Project

The Halm project specifically supports the above-mentioned gaps in real-time systems by integrating the three pillars of safety into one unitary system:

- **O1 – Smart Vision Module:** Edge-optimized computer vision of spatial hazard localization.
- **O2 – Smart Sense System:** Sensor fusion for instantaneous fall and impact detection.
- **O3 – Centralized Data Analytics Dashboard:** Multi-modal data fusion dashboard that integrates vision, motion, and GPS data to create a unified safety map.

Halm re-frames the occupational safety system paradigm by synthesizing these aspects as a shift in paradigm to an active and instantaneous detection approach beyond the constraints found in modern studies.

Table 1: Literature Review Synthesis Matrix

Safety Pillar / Focus	Methodology	Limitation / Gap Identified	Halm Objective	Relevance to Halm
Smart Vision & Environmental Awareness	Camera Surveillance & Live Broadcasting	Monitoring is Passive Surveillance; lacks autonomous, edge-based spatial hazard localization [2], [3] [10], [12], [16], [17], [18].	O1	Halm requires using computer vision (CV) as an autonomous digital safety overlay, not just a remote surveillance tool.
Smart Sense & Kinematic Analysis	Threshold-based Accelerometers / Gyroscopes	Systems are fundamentally Reactive; they only detect incidents after impact has occurred (post-facto detection) [4], [8], [9], [19], [20].	O2	Halm aims to achieve instantaneous detection to initiate the rapid, time-critical intervention required during a fall event.
Data Fusion & Situational Awareness	Separate Data Streams (IoT Dashboards)	Data platforms are Functionally Decomposed; lack true multi-modal fusion for a unified safety map [2], [3], [6], [9], [12], [21], [22].	O3	Halm will require a safe combination of visual and kinetic alerts on a single, unified dashboard to enable coordinated command-response actions.
Methodologically Advanced AI	Advanced AI (VLM, RNN/Forecasting)	Predictive AI/VLM models exist but are not integrated into practical, real-time smart helmet prototypes [2], [7], [11].	O1, O2	Halm focuses on edge-optimized AI and sensor fusion, aiming to integrate robust models into a wearable environment.

3 Scope and Deliverables

This section outlines the broad area, alignment, and key deliverables of the Halm project. It details the aims of the project, how it is related to the identity and the goals in society of King Faisal University, and tangible results expected in the implementation of the project. Taken together, these elements contribute to the overall understanding of the project development path, relevance, and anticipated outcome.

3.1 Definitions of the project's scope

Halm aims to improve the safety and productivity of construction site workers by applying smart helmet technology. The current subject area of the project includes a variety of issues related to worker safety, which will allow both workers and safety officers to make evidence-based choices that will lead to a safer and more efficient workplace. The scope of the project includes three major domains: smart vision for environmental hazard detection, which helps with detecting potentially hazardous areas, pits, and sharp tools in construction sites; smart sense for worker safety monitoring, which will detect falls and send direct emergency messages to safety officials; and real-time data integration by collecting and processing the data and displaying the information along with GPS locations through an interactive dashboard.

3.2 Alignment with KFU identity and societal impact

Halm is aligned with the identity of King Faisal University (KFU), which focuses on the areas of innovation, research, and community service. It specifically supports KFU's focus on research that emphasizes societal challenges and supports local priorities in the context of Vision 2030. While the university mainly focuses on food security and environmental sustainability, it also emphasizes applied technology and sustainable solutions to improve society [9], [13]. Furthermore, Halm aligns with the KFU's mission of attaining a future-oriented impact by contributing to key areas of concern through reducing workplace accidents and ensuring a safer working environment.

Halm directly contributes to societal impact by addressing important problems in construction sites that might affect worker safety and emergency response. It will reduce

the injuries on the construction site by detecting hazardous areas and issuing real-time alerts. In addition to safety at the workplace, it promotes a wider culture of safety and responsibility in sectors that are essential to the economy. Finally, the project acts as both a security measure for individuals and as a strategic tool that improves the resilience of society.

3.3 Expected project deliverables

The expected deliveries of this project will include hardware and software components to improve the safety and efficiency of workers, including three main elements:

- Smart Helmet prototype with built-in sensors, AI modules, and connectivity.
- Website Dashboard connected to the smart helmet that will provide safety officers with real-time visual analytics, GPS-based tracking, and send alerts to safety officers.
- Data reports from real-time monitoring will be available on the website for data analytics, safety audits, and decision-making.

4 Methodology

Halm will be developed based on the Agile approach, which allows for more rapid development, continuous testing, and adjustable optimization of the system components. This method allows ensuring that all features, including hazard detection and fall detection, are thoroughly tested, evaluated, and enhanced in short development cycles. The methodology is divided into a few discretized stages that ensure scalability, reliability, and efficiency.

4.1 Requirement Analysis

The stage entailed functional and non-functional requirement identification through literature review, benchmarking, and discussions with potential users, including construction safety engineers and site supervisors.

- **Stakeholder Requirement:** Construction workers and site managers were polled to establish realistic use cases, which include fall detection, hazard localization, and real-time emergency alerts.
- **Gap Identification:** An analysis of extant smart helmet prototypes showed a lack of real-time fall detection accuracy and limited multi-modal data fusion.
- **Functional Requirements:** System functions were described, such as Smart Vision that supports hazard localization, Smart Sense that supports motion and fall detection, and a centralized analytics dashboard.
- **Non-functional Requirements:** Performance (less than one second alert), usability (durable design, lightweight), scalability (supports multiple helmets in a network), and security (end-to-end data encryption) criteria were defined.
- **Prioritization:** Agile sprint planning was driven by classifying requirements based on risk criticality and feasibility of implementation.

4.2 System design and component architecture

Halm systems will be developed based on component-based architecture, which incorporates hardware and software modules that will be designed to be modular, scalable, and enable further expansion in the future.

- **Hardware Design:**
  - The Raspberry Pi 5, with 8GB of RAM, is the central processing unit in the hardware architecture and coordinates the flow of data between system modules.
  - The inertial measurement unit (IMU), specifically a BNO085 IC, is utilized to measure the linear acceleration and angular velocity, and hence fall and motion detecting, and a Raspberry Pi camera V2 module is used to record real-time video streams to detect visual hazards through computer-vision algorithms.
  - The T-SIM7670G (LILYGO) module provides connectivity and geospatial positioning and accommodates 4G LTE, GPS, and IoT protocols for transmitting data to a centralized dashboard.
  - A WS2812B RGB light will be used as a visual alarm to warn staff, and a UGREEN 10000 mAh power bank will be used to provide power to operate for a long time.
  - All constituent parts are covered safely in an industrial safety helmet and are connected by Dupont cables used to distribute power and passed signals.
- **Software Design:**
  - Python algorithms will be developed to perform kinematic detection based on IMU data using mathematical and statistical analysis.
  - Computer Vision will be merged with software to identify environmental risks based on previously trained datasets.
  - Developed a web-based dashboard to visualize real-time alerts, location data, and analytics.
- **Agile Integration:**

  Agile development will be adopted to divide the system into discrete sprints, where each component will be developed, tested, and refined based on the test results.

4.3 Implementation and Data Pipeline

The implementation phase will transform the design and develop the working prototype by assembling all hardware and software components and verifying efficient data flow and protection.

- **Hardware Integration:**
  - The hardware setup has a Raspberry Pi 5 (8GB) as the main central processing unit, with several other components, including the BNO085 IMU motion tracking unit in the category of inertial measurements and the Raspberry Pi Camera V2 hazard detection unit.
  - Additional modules, such as the T-SIM7670G (LILYGO), provide 4G, GPS, and IoT connectivity and can therefore be used to ensure reliable transmission to the cloud. The WS2812B RGB LED is used as a visual indicator of personnel.
  - A UGREEN 10000mAh power bank is used to supply power, and a Raspberry Pi Active Cooler is used to ensure thermal stability.
  - The industrial safety helmet has all components securely incorporated and is linked with Dupont wires through which signal and power routes are made.
- **Software and AI Development:**
  - Python code will be written to handle and process all sensor inputs, including IMU, camera, and GPS data streams.
  - Signal processing techniques will be applied to preprocess and filter IMU acceleration and gyroscope data to ensure clean, reliable kinematic features. A supervised learning model (Random Forest Classifier) will then be trained to classify motion patterns and distinguish normal worker movement from fall events.
  - Edge-optimized Computer Vision models (such as MobileNet-SSD or YOLOv5-Nano) will be trained to identify and localize hazardous objects and unsafe areas in real-time video frames.
  - A web-based interface will be developed using HTML, CSS, JavaScript, and supporting frameworks to visualize real-time alerts, GPS locations, and historical analytics on the centralized dashboard.

- **Data Management:**
  - Develop preprocessing pipelines to clean and align multimodal data (images, motion, and GPS).
  - Secure data storage with test data control, encryption, and auto backup.
- **Agile Workflow:**
  Each sprint generates a functional increment, starting with data collection, then AI training, and finally dashboard integration, ensuring continuous validation and improvement.

4.4 Detailed Description of the Specific Models

In terms of AI models, Halm will use two major model classes. For the Smart Vision module, we will use edge-optimized Computer Vision architectures, including MobileNet-SSD or YOLOv5-Nano, due to their suitability for embedded systems such as Raspberry Pi and their potential to perform real-time object detection of sharp tools, pits, and unsafe areas. This model will be trained using a custom curated dataset, which includes typical construction site hazards and real site objects that are taken from controlled sample environments. In the case of the Smart Sense module, IMU signals will be analyzed by a machine learning classifier; once acceleration and gyroscope data has been preprocessed, a supervised learning model will be applied to classify motion patterns to distinguish normal movement and sudden falls. The models will be repeatedly checked and refined during the Agile sprints to ensure accuracy and achieve an end-to-end alert latency of less than one second.

4.5 Data Collection and Dataset Preparation

This section outlines the datasets to be used to train and test the main Artificial Intelligence (AI) elements of the Halm. The data-collection plan will have high-quality, publicly available data, usable with edge-optimized models of computer vision (CV) and real-time kinematic classification and will attempt to reduce bias and to cover realistic construction site hazards in a representative way. Since the Halm system is based on a multi-modal fusion strategy, separate pipelines are created to connect the visual (Smart Vision - O1) and kinematic (Smart Sense - O2) data streams.

4.5.1 Smart Vision (O1) Datasets: Environmental Hazard Localization

The Smart Vision Module requires training data that supports the detection and localization of hazards in a construction site, including pits, holes, and sharp tools. Since each of the datasets is not a complete set that covers all the types of hazards in a helmet context, two specialized datasets are selected and combined into a unified training set.

4.5.1.1 Dataset 1: Small Construction Tools Dataset (Source: Dataset Ninja) [23]

- **Purpose:** The data to be used will be high-resolution images of small construction tools (e.g., cutters, hammers, and drills), which will be used to train the edge-optimized object-detection model (MobileNet-SSD or YOLOv5-Nano) that will detect sharp-tool hazards and equipment-related hazards.
- **Preparation:**
  - Extract the annotations of bounding boxes of tool categories that are relevant to construction safety.
  - Scale pictures to the Raspberry Pi Camera V2 resolution to make sure that it works correctly on the deployment.
  - Data augmentation (brightness variation, rotation, motion blur) can be used to replicate the conditions at a camera on a real site.
  - Unify labels into one hazard: Sharp Tool / Equipment Hazard.

4.5.1.2 Dataset 2: Annotated Potholes Dataset (Source: Kaggle) [24]

- **Purpose:** This data is an annotated collection of pothole pictures, uneven surfaces and road defects. These visual characteristics are closely related to the pits, unsafe depressions, and ground-level hazards that workers face, thus directly contributing to (O1) environmental detection requirement.
- **Preparation:**
  - Change the label of the pothole to a standardized one: (Pit / Unsafe Area).
  - Image dimensions and annotation formats are normalized to match the construction-tools dataset.
  - Set up both datasets into a single CV hazard dataset that consists of tools and pits.
  - Support a regular class mapping and dataset balancing to avoid bias in the model on any of the hazard types.

4.5.2 Smart Sense (O2) Dataset: Fall and Movement Classification

The Smart Sense System requires time-series data with accelerometer and gyroscope data of various human motions, such as transitions and sudden motions. This information is the basis of the Halm objective of instantaneous fall detection using high-frequency IMU signals.

4.5.2.1 Dataset 3: AGRO Human Activity IMU Sensor Dataset (Source: Kaggle) [25]

- **Purpose:** The data set has tri-axial accelerometer and gyroscope data recorded during the normal and fast human activities, and this offers appropriate motion patterns to train the random forest classifier that is utilized in fall detection.
- **Preparation:**
  - Pre-processing actions are like those of the BNO085 IMU, which guarantees that training and real-time execution are compatible.
  - Smooth IMU raw signals using noise and low-pass filtering.
  - Slide the time-series data in 2-seconds windows, which have a 50 percent overlap.
  - Capture some of the pertinent kinematic data (maximum acceleration magnitude, gyroscope magnitude, root mean square (RMS) values, and skew and kurtosis of acceleration signals).
  - Map labels of activity into binary classes: **Class 0: Normal Movement** and **Class 1: (Fall / Imminent Impact)**.
  - These measures will ensure that the classifier can differentiate between falls and normal movements of workers

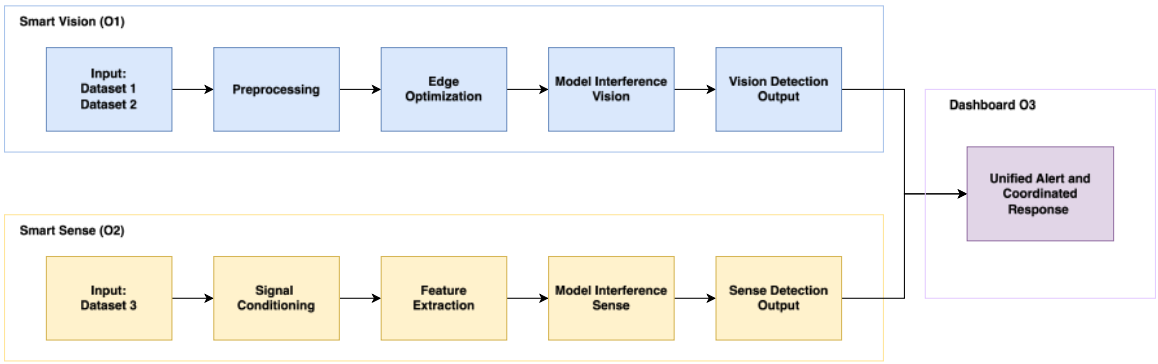

Figure 2: Multi-Modal Data Preprocessing and Edge Fusion Pipeline

The functional integration of the Halm system relies on continuous, parallel data processing pipelines to handle the heterogeneous nature of the sensor inputs. To visually summarize the implementation methodology, Figure 2 details the entire workflow, illustrating the separate data preparation processes for the Smart Vision and Smart Sense modules and how their high-fidelity outputs converge at the edge-based processor (Raspberry Pi 5) to form the Multi-Modal Data Fusion decision.

4.6 Data Management and Data Security

To ensure the reliability of the systems and ensure user privacy, processing of all datasets is done according to the principles of secure data management:

- The encrypted data is put in controlled directories that contain processed data.
- Labeling and annotations are uniformed to support multi-modal data fusion between O1 and O2.
- Data segregation Test data separation is implemented to avoid data leakage and to achieve valid performance benchmarking.

4.7 Testing and Validation

Extensive testing and validation will be necessary to ensure that the Halm prototype meets performance, reliability, and usability goals, and multiple iterative cycles will be performed under controlled and simulated conditions.

- **Unit Testing:** The system will be tested to confirm the functionality of all hardware items (sensors, camera, GPS) and software (AI models, data transfer, dashboard interface).

- **Integration Testing:** Integration testing will check the communication between modules to make sure that data can be easily sent between the helmet and the cloud and dashboard.
- **Performance Testing:** Latency of the measured system, with a goal of less than one second alerting reaction time and low levels of packet loss during wireless transmission.
- **Validation:**
  - Compare fall detection results and ground-truth datasets with respect to accuracy.
  - Continuous Improvement: Agile retrospective meetings will be used to determine performance bottlenecks and apply improvements to the next sprint cycle.

4.8 Future Development

Future efforts will concentrate on increasing system intelligence, scaling, and usability of the system to guarantee long-term sustainability and applicability.

- Natural Language Processing (NLP) analyzes past safety reports and enhances predictive accuracy.
- Development of fall prediction algorithms to anticipate hazardous movements.
- Integration of physiological monitoring (heart rate and temperature) to assess worker fatigue and heat stress.
- Investigation of energy optimization wearable solutions to increase battery life and reduce power consumption.

Through real-time recognition of dangers and the delivery of exposure alerts, Halm system significantly improves on-site safety, thus helping to prevent accidents and create a safer and more effective construction environment in the future.

5 Project Requirements

Halm requirements are divided into two main groups: functional requirements, which highlight the primary features of the project and system actions, and non-functional requirements, which highlight the quality attributes such as performance, security, and usability. Together, they ensure that the smart helmet system will achieve its leading role in enhancing construction site safety and providing a clear framework for the development and evaluation of this project.

5.1 Functional Requirements

- **Spatial Hazard Localization:** Recognize and map hazardous items or the workplace at construction sites (O1).
- **Kinematic Detection Analysis:** Detect worker falls with IMU information (O2).
- **Real-Time Data Transmission:** Linearly feed processed sensor information and notifications to the central computer server.
- **Security of Access:** Safety officers should be able to log in to the site through authorization measures.
- **Unified Situational Dashboard:** Give a single, interactive dashboard that shows a multi-modal data combination and visual analytics (O3).
- **Real-time Alerting:** Send automated, threshold-based alerts to safety officers in real time through the web.
- **Location Tracking:** Real-time GPS location tracking of personnel and assets.
- **Reporting & Auditing:** Produce analytic data reports through an evaluation and summary of stored safety data.

5.2 Non-Functional Requirements

- **Performance:** The alert time of critical events (hazard localization, fall detection) should not exceed one second to ensure operational efficiency.
- **Security:** The transmission and storage of data between helmets, dashboards, and servers should be encrypted (end-to-end) to preserve the privacy and integrity of worker data.

- **Usability:**
    - The smart helmet must be light and comfortable to be used throughout the entire working hours with long working shifts.
    - The battery life should be able to last the entire system's functioning duration of a complete work shift.
    - The interface should provide intuitive and easy navigation for safety officers.
- **Scalability:** The system architecture should be scalable to accommodate hundreds of helmets at construction sites on a real-time basis.
- **Reliability:** the system should have fault tolerance provisions and should be capable of maintaining service in the case of high-risk safety conditions to forestall an abrogator termination of services.
- **Data Integrity:** Data transmissions between the helmet and the central system should be correct and have a low packet loss.
- **Availability:** There should be a minimum up time for the central monitoring website of at least 99% of the operating time.
- **Maintainability:** The internal firmware components of the helmet, as well as the detachable computer-like software components of the website, must allow easy updates.

6 Design and Analysis

The Design and Analysis section is the core technical blueprint for the Halm project, detailing how the system's safety objectives are met. It starts by outlining the System Architecture and Design Justification, explaining the modular design and the roles of key components like the Raspberry Pi 5 for edge processing, the BNO085 IMU, and the T-SIM7670G module. This is followed by defining the System Components and Interaction Flow, which details the real-time data pathway from the helmet sensors to the central dashboard. The System Flow further elaborates on the dual-path logic (IMU and camera) for risk detection, alert generation, and data transmission. The entire section is visualized and clarified through essential diagrams, including the System Architecture, Flowchart/data flow, database design, use case, and hardware connection diagrams.

6.1 System Architecture and Design Justification

The architecture of Halm is modular and layered, which ensures ease of scaling, reliability, and responsiveness in real-time, all of which are critical to industrial safety applications. Its design focuses on a smooth combination of sensing, processing, and analytics subsystems to ensure effective data flow and timely hazard mitigation.

At the input layer, the system integrates multiple sensing modules that continuously collect complementary data sources. The IMU sensor provides real-time motion and fall detection using the kinematic analysis (Smart Sense System), whereas the Raspberry Pi Camera V2 provides the visuals of the environment to localize hazards using trained computer-vision algorithms (Smart Vision Module). Collectively, these elements make it possible to see internal worker motions and external risks at the workplace at the same time.

The processing layer is centered on the Raspberry Pi due to its powerful computing capabilities and the ability to be used with embedded AI programs. Running algorithms on the Raspberry Pi locally leads to the elimination of remote server usage and allows cutting latency by a factor, ensuring sub-second response times. This edge-based processing architecture plays a crucial role in time-sensitive situations in which prompt actions can prevent severe injury.

The T-SIM7670G module handles connectivity and geospatial tracking with a 4G-LTE and GPS transmission. This guarantees secure real-time communication between the helmet and cloud infrastructure, which enables the synchronization of data continuously. Also, the WS2812B RGB LED serves as an immediate local warning device, providing on-site feedback in areas where network connectivity might be weak.

At the output layer, all data streams, IMU readings, visual detections, and GPS coordinates are transmitted to a centralized data analytics dashboard. This information is summarized on the dashboard into a single visualization that facilitates situational awareness as well as decision-making by the safety officers. The system helps to reduce the fragmentation characteristic of previous smart-helmet systems, as it integrates all safety incidents into one interface, which improves the process of coordination during emergency response.

The reason why this architectural design was selected was to balance integration, speed, scalability, and security. Multimodal sensing on a common platform would allow multi-sensory monitoring; edge sensors can be used to deliver quick and dependable notifications; hardware can be modular, such that it is easy to deploy in large numbers on this system; and secure data communication would ensure system integrity. The design decisions, in turn, substantiate the architecture as a superior solution towards the real-time and proactive worker protection in the construction and industrial settings that was the goal of Halm.

6.2 System Components and Interaction Flow

The Halm system is based on a highly integrated, component-based system that is designed to deliver the workers a smart helmet. The brain of this architecture is a Raspberry Pi 5 (8 GB) that serves as the central processing unit, manages the flow of data, and carries out all artificial intelligence (AI) and kinematic detection logic. Two main sensor systems are used for data acquisition: the Smart Sense System and the Smart Vision Module. The first one uses a BNO085 IC Inertial Measurement Unit (IMU) and Python-based mathematical analysis for motion monitoring to facilitate real-time fall detection. The second one uses a Raspberry Pi Camera V2 and a trained computer vision model to autonomously detect and localize hazards in the environment in real-time video streams.

To support connectivity, location tracking, and secure communication, the system uses a T-SIM7670G (LILYGO) module, which has 4G LTE, GPS positioning, and Internet-of-Things protocols. This carefully chosen hardware ensures that important data is captured, processed, and delivered accurately.

The interaction flow of the critical system begins with the continuous acquisition of sensor and GPS data by the components of the helmet. The Raspberry Pi 5 uses this multi-modal data directly at the edge, therefore allowing immediate responses to meet the performance goal of sub-second alerting. When a critical event is detected (e.g., a fall or a localized hazard), the system automatically triggers a local WS2812B RGB light, which immediately causes the worker and other personnel in the vicinity to see a visual alert. At the same time, the processed alert is sent safely to the secure cloud server through the end-to-end encrypted channel by the T-SIM7670G with the addition of the real-time GPS position of the worker. The multi-modal data received is then combined and shown in the web-based centralized dashboard. This final output provides safety officers with a unified situational awareness platform, therefore enabling effective and quick response to risks.

6.3 System Flow

The System Flow delineates the real-time, event-driven operational logic that governs the Halm system, providing the precise mechanism for achieving instantaneous risk detection and coordinated response. This process is characterized by continuous, parallel data processing on the edge (the Raspberry Pi 5), which is essential for bypassing latency issues and meeting the sub-second alert goal. The flow commences with the simultaneous monitoring of worker kinematics and the external environment, realized through two distinct and independent pathways that run concurrently once monitoring is active.

6.3.1 Dual-Path Sensing and Edge Detection

The system operates two critical, parallel detection branches on Raspberry Pi 5, relying on edge computing to minimize response time:

- **Smart Vision Module (O1):** Concurrently, the Raspberry Pi Camera V2 captures real-time video streams. These streams are processed by edge-optimized Computer

Vision models running on the Raspberry Pi. This process autonomously detects and classifies environmental hazards like pits or sharp tools. If a hazard is detected, this path also initiates the unified alert sequence.

- **Smart Sense System (O2):** The BNO085 IMU continuously streams movement data. This data is analyzed by kinematic detection algorithms (Python-based mathematical analysis) to detect motion patterns indicative of imminent falls (e.g., loss of balance). If a fall is detected, this path triggers the unified alert sequence.

6.3.2 Unified Alert and Coordinated Response

When either detection path confirms a critical event (a fall or a hazard), they converge to execute a rapid, coordinated response:

- **Local Intervention:** The system immediately generates an alert for the worker and triggers the WS2812B RGB light on the helmet, providing an instantaneous visual alert.
- **Secure Data Transmission:** Simultaneously, the T-SIM7670G module retrieves the worker's GPS location and securely transmits the processed, multi-modal alert data (including location, alert source, and type) via the 4G/LTE channel to the cloud backend.
- **Safety Officer Benefit (Situational Awareness):** The cloud backend updates the central database with the time-stamped incident data. This alert, integrated with the real-time GPS coordinates, is instantly displayed on the Centralized Data Analytics Dashboard (O3). This unified interface provides safety officers with a holistic view of risks, enabling swift, informed command-response actions and enhancing overall situational awareness.

6.4 Diagrams

6.4.1 Database Design Diagram

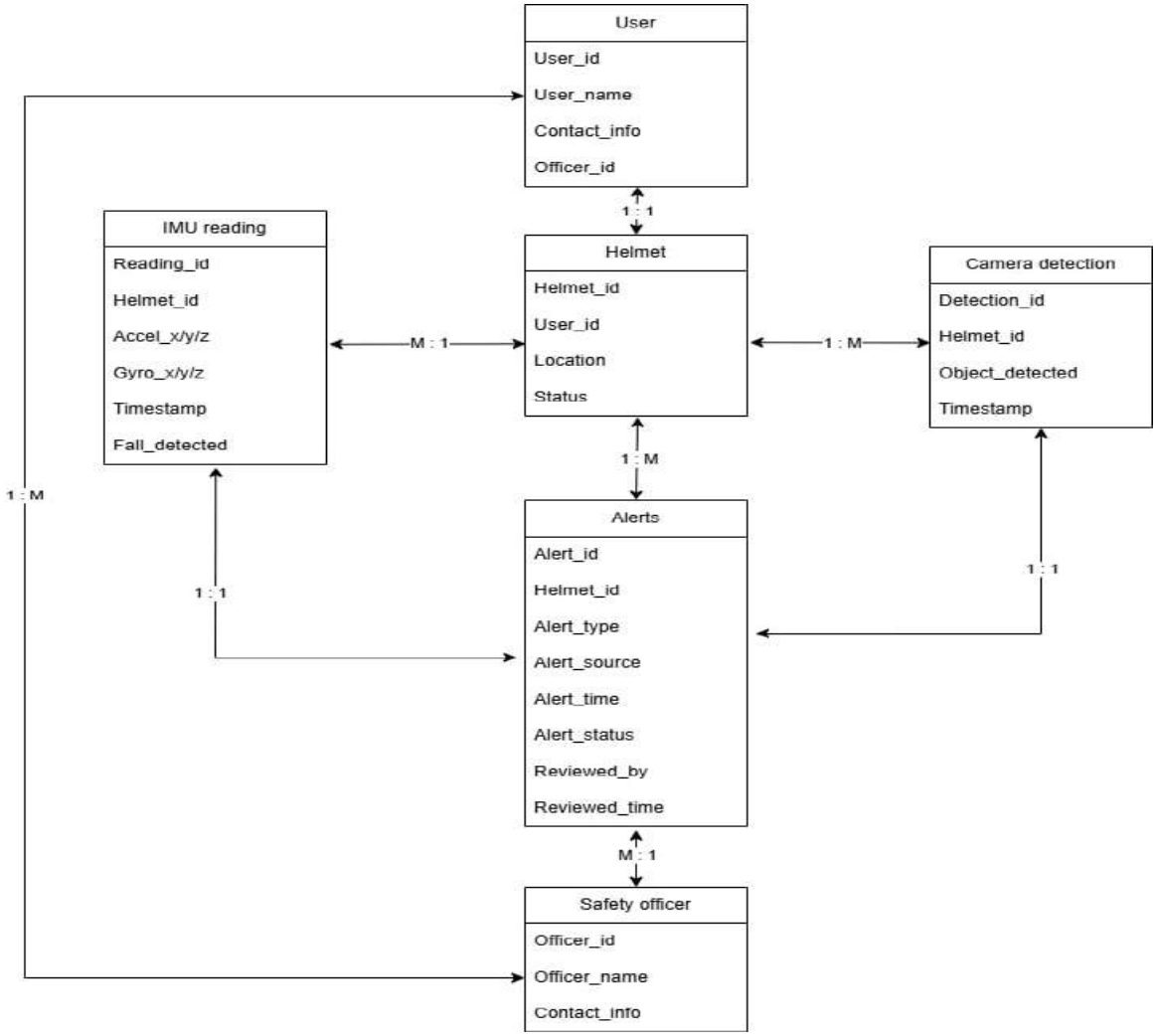

Figure 3: Database Design Diagram for the Halm Smart Helmet.

Entity-relationship diagram of the safety-monitoring system showing users, helmets, IMU readings, camera detections, alerts, and safety officers. A user can only have one helmet and can produce numerous IMU readings, camera detections, and alerts. The alerts originate from either IMU or camera sources. Safety officers have various users under their supervision and can have various alerts.

6.4.2 System Flowchart

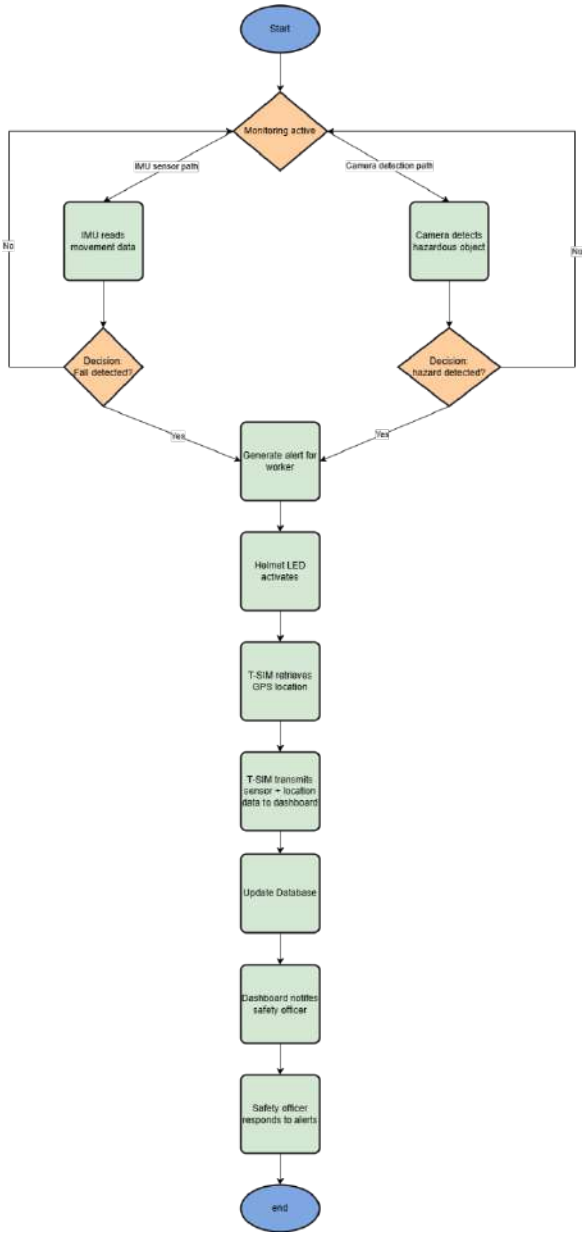

Figure 4: System Flowchart for the Halm Smart Helmet.

Flowchart of the monitoring process showing how the smart helmet detects worker falls and environmental hazards using IMU sensors and a camera module. When a fall or a hazardous object is detected, the system triggers a visual alert using the LED-illuminated helmet and sends the relevant information to the central dashboard using the T-SIM module. After this, the dashboard saves the information in the database and notifies the safety officer, who, in turn, examines it and responds to the incident.

6.4.3 System Architecture Design

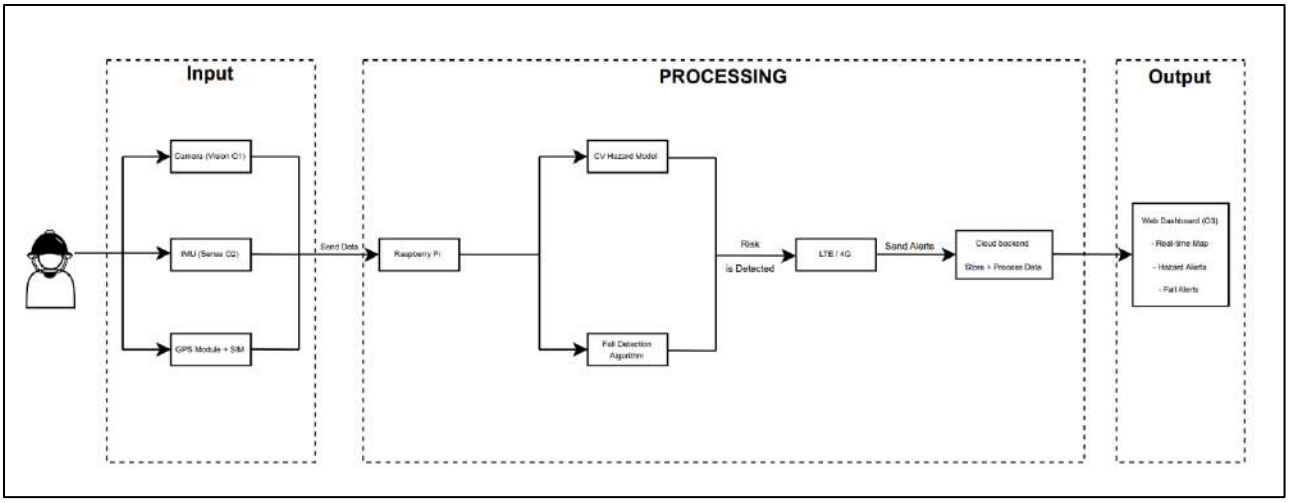

Figure 5: System Architecture Diagram for the Halm Smart Helmet.

This architecture illustrates the overall end-to-end workflow of the Halm system across the three major layers: input, processing, and output. In the input layer, raw sensory signals are gathered from the helmet components, including the camera module (Smart Vision - O1), the inertial measurement unit (Smart Sense - O2), and the GPS/SIM module. These data streams are then sent to the Raspberry Pi, which is the central controller. In the processing layer, AI models are executed on the Raspberry Pi, where the computer vision model performs hazard detection, and the fall detection algorithm identifies risk events. After the risk has been detected, the integrated results are sent via LTE/4G connectivity to the cloud backend. In the output layer, the cloud backend receives incoming updates, processes them, and sends them to the centralized web dashboard (Smart Analytics - O3), where the safety officers can see real-time location tracking, hazard alerts, and fall alerts in a unified interface. In this workflow, we are showing how Halm integrates multi-modal sensing, edge AI, and cloud analytics to provide proactive worker protection.

6.4.4 Hardware Connection Diagram

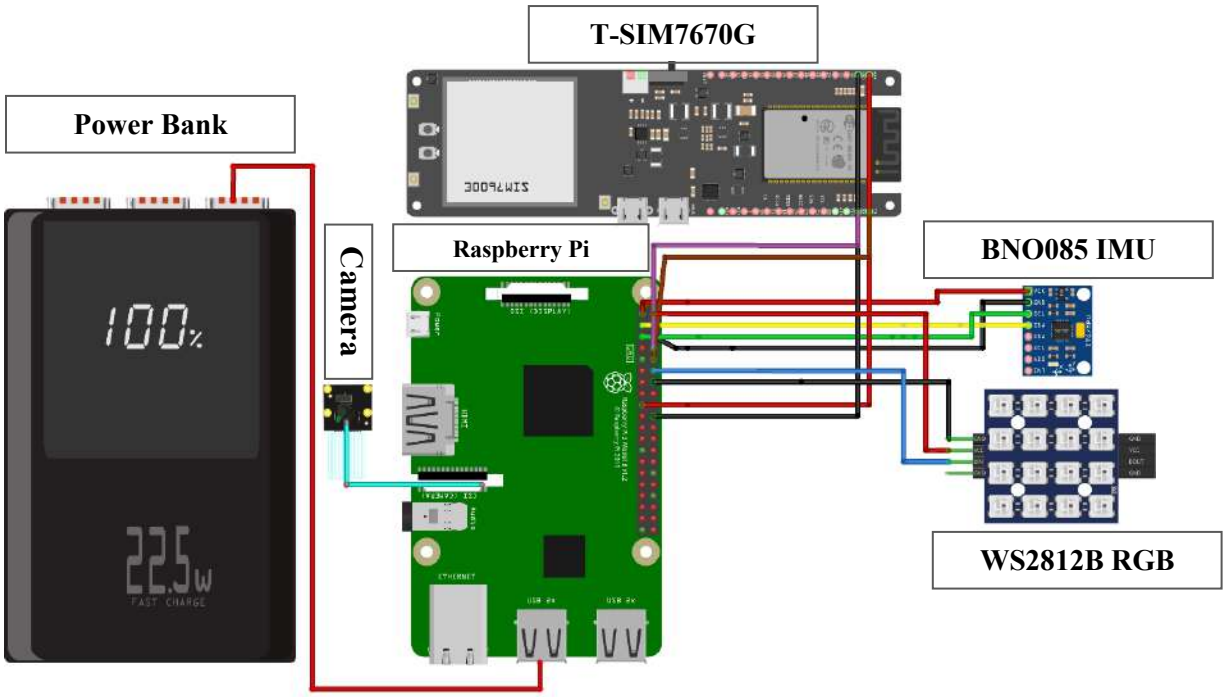

Figure 6: Hardware Connection Diagram for the Halm Smart Helmet.

Figure 6 shows a Hardware Connection Diagram that provides the detailed, pin-level blueprint for assembling the Halm prototype, demonstrating how the system's components are interconnected to ensure low-latency communication and robust power management within the helmet, prioritizing the use of the Raspberry Pi 5. The system is compartmentalized into sensing, communication, and output modules, with all connections channeled through the Raspberry Pi 5's 40-pin GPIO header. Specifically, the Smart Sense System (O2) BNO085 IMU utilizes the I²C bus for efficient data transfer, connecting to the dedicated SDA and SCL pins and drawing power from a reserved 3V3 power pin (Pin 1). The T-SIM7670G (Connectivity) modem interfaces using the reliable UART protocol (TX/RX), connecting to the dedicated hardware serial pins (GPIO 14/15) and drawing power from a separate 3V3 power pin (Pin 17). The Smart Vision Module (O1) Raspberry Pi Camera V2 is connected using its Flexible Flat Cable (FFC) directly to the dedicated CSI Port on board, which provides the necessary high bandwidth. Finally, the Local Alert (WS2812B) addressable LED light is connected to a 5V power pin for adequate brightness and is controlled via a standard GPIO pin (**GPIO 18**), utilizing a PWM or single-data-line protocol for visual alerts.

1.4.4.1 Halm Hardware Connection Diagram Summary

The following table details the specific pin assignments and protocols for verifying and assembling the Halm smart helmet prototype:

Table 2: Hardware Connection Diagram for the Halm Smart Helmet.

Component	Component Pin Name	Raspberry Pi Pin Name / Number	Protocol / Function	Standard Wire Color
BNO085 IMU (Smart Sense O2)	VCC	3V3 Power (Pin 1)	Power for IMU	Orange
	GND	GND (Pin 9)	Ground Reference	Black
	SDA	GPIO 2 (Pin 3)	I²C Data Line	Yellow
	SCL	GPIO 3 (Pin 5)	I²C Clock Line	Green
T-SIM7670G (Connectivity)	3V3	3V3 Power (Pin 17)	Power for Modem	Red
	GND	GND (Pin 25)	Ground Reference	Black
	TX (Transmit)	GPIO 15 (Pin 10)	UART Receive Data (RPi RX)	Brown
	RX (Receive)	GPIO 14 (Pin 8)	UART Transmit Data (RPi TX)	Purple
WS2812B RGB Light (Alarm)	VCC	5V Power (Pin)	Power for LED	Red
	GND	GND (Pin 20)	Ground Reference	Black
	Data In (DIN)	GPIO 18 (Pin 12)	PWM / Single Data Line	Blue
Raspberry Pi Camera V2	N/A	CSI Port	High-Bandwidth Video Stream	N/A (Flat Cable)
Power Bank	N/A	USB-C Input Port	Main Power Supply	N/A (USB-C Cable)

6.4.5 Use Case Diagram

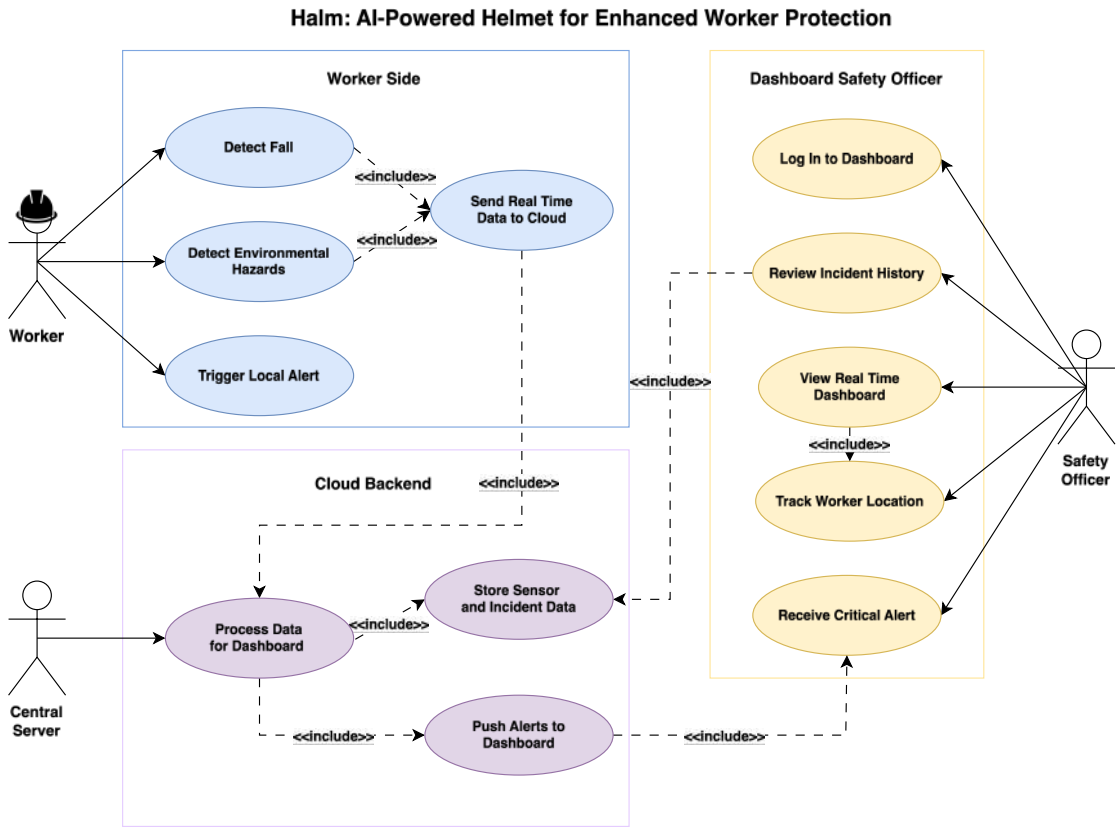

Figure 7: Use Case Diagram

Figure 7 above illustrates a use case diagram that defines the functional scope and the interaction points for the Halm system across its three architectural layers: the **Worker Side (Edge Processing)**, the **Cloud Backend (Data Fusion)**, and the **Dashboard Safety Officer (Centralized Data Analytics)**. It maps how critical events, such as Detect Fall and Detect Environmental Hazards, initiate the mandatory process of data transmission to the cloud. The <<include>> relationships ensure that all detection events are immediately propagated through the backend to the safety officer for a unified, rapid response.

7 Technical Development Plan

This section outlines the technical development plan used in designing and implementing the Halm Smart Helmet. It begins with the strategic technical decisions that define the system architecture, including the choice of core hardware, Artificial Intelligence (AI) models, and software frameworks necessary to process real-time and multimodal data. The comparative analysis of alternative solutions that follows justifies the chosen design and confirms its superior capabilities in scalability, responsiveness, and built-in safety intelligence. Altogether, this section documents the entire engineering process, from component selection to the development schedule, ensuring the Halm project is fully justified and solidly engineered to overcome the practical limitations of the industrial environment.

7.1 Technical Choices and Justification of Their Selection

The technical architecture of the Halm project was specifically chosen to overcome the drawbacks of functional fragmentation and reactive detection of the current safety solutions. The core architecture is designed as an edge-based processing system, prioritizing speed, integration, and high data fusion.

7.1.1 Edge Processing Unit: Raspberry Pi 5 (8GB)

- **Choice:** The Raspberry Pi 5 was selected as the central processing unit to run AI algorithms at the edge (locally).
- **Justification:** This approach eliminates the need for remote servers for initial processing, which largely eliminates latency. The goal is a sub-second reaction period for critical events like hazard or fall detection. This high-speed, localized response is crucial for time-sensitive situations in industrial applications, where rapid intervention can prevent serious injuries.

7.1.2 Smart Sense Sensor: BNO085 (IMU)

- **Choice**: The BNO085 Inertial Measurement Unit (IMU) was chosen for kinematic analysis.
- **Justification:** Selecting a high-performance sensor ensures the reliability needed for instantaneous fall detection (O2). BNO085 provides high accuracy and pre-fused motion data (acceleration and angular velocity), allowing the system to detect fine-tuned motion patterns indicating impending falls or loss of balance, thereby enabling prompt intervention and minimizing the severity of an incident.

7.1.3 Smart Vision Sensor: Raspberry Pi Camera V2 Module

- **Choice**: The Raspberry Pi Camera V2 module was chosen to capture visual information.
- **Justification:** Connected via its high-bandwidth CSI port to the Raspberry Pi 5, this camera reliably records real-time video streams, which are necessary for processing and hazard localization. This enables the Smart Vision Module (O1) to autonomously detect and position environmental hazards, such as pits and sharp tools.

7.1.4 Connectivity and Location: T-SIM7670G (LILYGO)

- **Choice**: The T-SIM7670G (LILYGO) module was selected to provide communication and localization services.
- **Justification:** Based on 4G LTE, GPS, and IoT protocols, the module guarantees secure and real-time interaction between the helmet and the cloud infrastructure. It provides the real-time geographic location of the worker, thereby enabling the integrated situational-awareness dashboard (O3).

7.1.5 Hazard Localization AI Models (Smart Vision)

- **Choice**: Edge-optimized computer-vision architectures were selected. Specifically, MobileNet-SSD or YOLOv5-Nano.

- **Justification:** These models are highly compatible with embedded systems like Raspberry Pi. They offer the necessary compromise between computational speed and detection accuracy for dangerous objects (e.g., sharp tools, pits), directly supporting the function of the Smart Vision Module (O1).

7.1.6 Local Notification System: WS2812B RGB Light

- **Choice:** The WS2812B RGB light strips were integrated into the helmet.
- **Justification:** This illumination system serves as an effective instant local warning mechanism, providing immediate visual alerts to the worker and other individuals in the vicinity. This type of on-site alert is crucial during real-time events, especially in areas where network connectivity might be weak or delayed.

7.2 Comparison Of Alternative Solutions

The table provided compares the suggested Halm system and three other safety monitoring technologies: IoT-based Smart Helmet and Jacket System, Smart Safety Glasses, and a Safety Smartwatch. The solutions are considered in terms of important characteristics to identify their suitability for safety applications.

The Halm helmet is distinguished by its primary focus on unified, multi-modal safety (Vision, Kinematics, Fusion) and its high scalability architecture designed to support hundreds of helmets. The system aims for less than one second of low latency to detect major events. The IoT system and safety glasses offer real-time monitoring of health/environment and built-in PPE functions. They exhibit moderate scalability and higher or variable latency. The smartwatch is optimized for personal physiological and environmental monitoring. There is a wide difference between hardware and software integration. The Halm helmet is built using a Raspberry Pi 5 that uses special modified Python CV/IMU algorithms, which are expected to have high accuracy and performance with 8GB of RAM. Alternatives are based on microcontrollers, including ESP32, or software-based apps. As shown in Table 3, the Halm system's key advantage is its high integration, anticipatory safety, and low-latency target, positioning it as a potentially superior safety solution compared to the other options.

Table 3: Comparison of Alternative Solutions.

Feature	Halm: AI-Powered Helmet (Proposed)	IoT-based Smart Helmet and Jacket System [2]	Smart Safety Glasses [22]	Safety Smartwatch [22]
Primary Focus	Unified, anticipatory multi-modal safety (Vision, Kinematics, Fusion)	Real-time health, environment monitoring, and fall/helmet compliance	Integrated into PPE, general safety functions (e.g., location, vision)	Personal physiological and environmental monitoring
Scalability	High, component-based architecture designed to support hundreds of helmets	Moderate, built on ESP32/ThingSpeak, scalable but dependent on Wi-Fi infrastructure	High, easily deployable as a small, head-worn WSD	High, widespread consumer device with existing network infrastructure
Latency	**Target: < 1 second** alert time for critical events (hazard/fall detection)	**< 2 seconds** mean time from sensor to cloud dashboard	Variable, dependent on embedded processing or cloud communication	Variable, optimized for physiological data, but the latency for safety features can vary
Cost	Moderate (uses Raspberry Pi 5, Computer Vision, and IMU)	Moderate (uses off-the-shelf sensors and ESP32)	Moderate-to-High (specialized PPE integration)	Moderate-to-High (depends on model and industrial-grade features)
Software Integration	Multi-modal data fusion, specialized Python CV/IMU algorithms, unified web dashboard	Threshold-based C/C++ logic, ThingSpeak cloud visualization/alerts	Focus on specific application software, data streaming to the cloud	App-based software, often for physiological metrics and notifications
Hardware Integration	Raspberry Pi 5 (8GB), BNO085 IMU, T-SIM7670G (4G/GPS), embedded in a helmet	ESP32, BNO085 IMU, Pulse, DHT11, MQ Gas, Vibration, IR sensors; integrated into a helmet/jacket	Integrated into the safety glasses frame (e.g., camera, location, small sensors)	Wrist-worn, integrated sensors (HRM, GPS, accelerometer, etc.)
Accuracy	Expected High performance due to the usage of Computer Vision for spatial localization and instantaneous IMU for falls)	Moderate-to-High (92% fall detection sensitivity, reliance on DHT11 for environment)	Highly dependent on the quality and type of embedded sensors	High for core functions (HRM, steps), accuracy for construction-specific hazards is unverified
Pros	Highly integrated, anticipatory safety, multi-modal data fusion, low-latency target, specific spatial hazard localization	Comprehensive monitoring (health, gas, movement), proven implementation	Discreet, less intrusive than a jacket or large helmet, directly integrated with vision	High user acceptance, continuous physiological monitoring, and proven widespread use
Cons	High complexity, reliance on high-power processing (Raspberry Pi 5), framework still needs validation on live projects	Requires both a helmet and a jacket, limited predictive capability, lower quality environmental sensors (DHT11), and reliability dependent on Wi-Fi	Less capacity for large sensor arrays (e.g., gas, complex kinematics) due to size constraints, potential for vision obstruction	It may be less reliable for construction-specific environmental and proximity detection and does not support hazard detection.

7.3 Details Of Tools, Programming Languages, and Platform Used

7.3.1 Hardware Components and Embedded Systems

Table 4: Core Hardware Components and Embedded Systems

Component / Tool Name	Primary Use in Halm Project
Raspberry Pi 5	Central Processing Unit (CPU) for **edge processing**, coordinating the system, and running **AI and data fusion algorithms**.
BNO085	High-fidelity IMU for the **Smart Sense System (O2)**, enabling instantaneous fall and kinematic detection.
Raspberry Pi Camera V2	Captures real-time video streams for the **Smart Vision Module (O1)** to localize environmental hazards.
LILYGO T-SIM7670G-S3 (4G LTE)	Provides 4G LTE, GPS, and IoT protocols for secure, real-time data transmission and location tracking (O3).
WS2812B RGB Light	Local visual alarm/indicator for immediate, on-site alerts to the worker.
RPi 5 Active Cooler	Ensures thermal stability and sustained performance of Raspberry Pi 5 under continuous AI workload.

7.3.2 Programming Languages and Operating Systems

Table 5: Programming Languages and Operating Systems

Language or Platform	Primary Use in Halm Project
Python	Core programming language for kinematic detection algorithms, sensor data handling, and running Computer Vision logic.
HTML, CSS, JavaScript	Used to build the secure, interactive Centralized Data Analytics Dashboard (O3).
Linux (Raspberry Pi OS)	The underlying operating system is required to deploy and execute Python and AI scripts for continuous operation.
C/C++	It may be utilized for low-level configuration and driver implementation for the IMU and connectivity module.

7.3.3 Software and AI Models

Table 6: Software Components and Artificial Intelligence Models

Software / AI Model	Primary Use in Halm Project
MobileNet-SSD or YOLOv5-Nano	AI architecture is used for real-time object detection of environmental hazards (O1) directly on the Raspberry Pi.
Supervised Learning Model (Random Forest (RF))	Analyzes preprocessed IMU data to classify motion patterns, distinguishing normal movement from sudden falls (O2). RF is selected for its high accuracy and rapid inference speed on Raspberry Pi 5, supporting the critical instantaneous detection requirement.
IoT Cloud Platform (Custom/External)	Securely receives, stores, and processes multi-modal data streams for presentation on the dashboard.

7.3.4 Design and Diagramming Tools

Table 7: Technical Design and Diagramming Tools

Logo	Design Tool	Primary Use in Halm Project
	Draw.io	Used to create complex diagrams like the Database Design Diagram and Use Case Diagram.
	Fritzing	A dedicated tool for creating the detailed circuit layout and visualizing the Hardware Connection Diagram (Figure 5).
	Canva	Used to create professional visual aids, graphical elements, and presentation materials. Specifically, it is used to create the Gantt chart.
	onlinegantt	Used to generate the project's visual timeline, or Gantt chart, for task management and scheduling.

7.3.5 Research Platforms

Table 8: Research and Academic Platforms

logo	Platform / Tool Name	Primary Use in Halm Project
	Google Scholar	Used for a comprehensive literature review and identifying influential studies.
	RABBITResearch	Used for generating the Bibliometric Co-Authorship Network (Figure 1) for literature review.

7.3.6 Collaboration Tools

Table 9: Project Collaboration Platforms

logo	Platform	Primary Use in Halm Project
	Notion	Central platform for organizing documentation, tracking tasks, managing research data, and internal team collaboration.
	Google Drive	Used for file sharing, version control, and storing all research files and reports securely.
	Google Meet	Used for holding regular remote team meetings for planning, progress review, and supervision.
	Microsoft Word	Used for drafting, editing, and finalizing the main academic report document.
	Microsoft PowerPoint	Used for creating Halm's presentation slides for defense and review.

7.4 Initial Algorithm (Pseudocode)

SECTION 1: SYSTEM INITIALIZATION AND CONSTANTS

```
FUNCTION INITIALIZE_HALM_SYSTEM ()
```

// O2: Smart Sense (Kinematics) Setup

```
SENSOR_IMU_BNO085 = CONNECT_I2C(I2C_BUS)
RF_FALL_MODEL = LOAD_MODEL("RandomForest_FallClassifier.pkl")
//Supervised Learning Model
WINDOW_SIZE_SEC = 2.0     // Sliding window size for feature extraction
SAMPLING_RATE_HZ = 50     // BNO085 data rate
WINDOW_SAMPLES = WINDOW_SIZE_SEC * SAMPLING_RATE_HZ
IMU_BUFFER = EMPTY_QUEUE ()// Circular buffer for time-series data
```

// O1: Smart Vision (Hazard Localization) Setup

```
CAMERA_MODULE = CONNECT_CSI(CSI_PORT)
CV_HAZARD_MODEL = LOAD_MODEL("YOLOv5-Nano_HazardDetector.pt")
// Edge-optimized CV model
CONFIDENCE_THRESHOLD = 0.70 // Minimum confidence for hazard detection
```

// O3: Connectivity & Alert Setup

```
COMMS_SIM7670G = CONNECT_UART(T_SIM7670G)
LED_LOCAL_ALERT = CONNECT_GPIO(WS2812B_RGB)
CLOUD_BACKEND_URL = "https://halm.cloud-analytics.kfu.sa/api/alert"
// Secure API Endpoint
```

// Start parallel monitoring threads

```
START_THREAD(MONITOR_FALL_RISK)
START_THREAD(MONITOR_ENVIRONMENT_HAZARDS)
END FUNCTION
```

SECTION 2: SMART SENSE (O2) - KINEMATIC DETECTION THREAD

```
FUNCTION MONITOR_FALL_RISK ()
WHILE SYSTEM_ACTIVE:
```

// Step 1: Data Acquisition & Buffer Update

```
IMU_DATA_POINT = READ_SENSOR(SENSOR_IMU_BNO085)
// Data: Accel_x, Accel_y, Accel_z, Gyro_x, Gyro_y, Gyro_z
IMU_BUFFER.ADD(IMU_DATA_POINT)
```

// Step 2: Edge Processing (Run only when a full window is available)

```
  IF IMU_BUFFER.SIZE() >= WINDOW_SAMPLES:
        CURRENT_WINDOW = IMU_BUFFER.GET_LATEST(WINDOW_SAMPLES)
```

// Step 2a: Feature Extraction

```
    FEATURES = EXTRACT_KINEMATIC_FEATURES(CURRENT_WINDOW)
    // Features include Max_Acc_Mag, Mean_Gyro_Mag, Accel_Skewness,...
```

// Step 2b: Classification (Machine Learning Model)

```
    FALL_PREDICTION = RF_FALL_MODEL.PREDICT(FEATURES)
    // Output: 1 (Fall/Imminent Impact) or 0 (non-fall)
```

// Step 3: Alert Triggering

```
        IF FALL_PREDICTION == 1:
            TRIGGER_UNIFIED_ALERT ("Fall_Detected", "Smart Sense O2")
        END IF
```

// Step 4: Slide Window (50% Overlap for real-time continuity)

```
IMU_BUFFER.REMOVE_OLDEST (WINDOW_SAMPLES / 2)
    END IF
    WAIT_FOR_NEXT_SAMPLE () // Maintain 50Hz sampling rate
END WHILE
END FUNCTION
```

SECTION 3: SMART VISION (O1) - ENVIRONMENTAL DETECTION THREAD

```
FUNCTION MONITOR_ENVIRONMENT_HAZARDS ()
WHILE SYSTEM_ACTIVE:
```

// Step 1: Data Acquisition (Real-time video frame)

```
CURRENT_FRAME = READ_VIDEO_FRAME(CAMERA_MODULE)
```

// Step 2: Edge Processing (Object Detection)

```
// Detects: pits, sharp tools, unsafe areas
DETECTIONS = CV_HAZARD_MODEL.INFER(CURRENT_FRAME)
```

// Step 3: Localization and Alert Check

```
FOR EACH DETECTION IN DETECTIONS:
        IF DETECTION.CONFIDENCE >= CONFIDENCE_THRESHOLD AND
DETECTION.LABEL IS IN ["Pit", "SharpTool", "UnsafeArea"]:
            // Create a summary of the localized hazard
            HAZARD_INFO = {LABEL: DETECTION.LABEL, CONFIDENCE:
DETECTION.CONFIDENCE, LOCATION_BOX: DETECTION.BOUNDING_BOX}
```

// Step 4: Alert Triggering (Pass hazard specifics to the unified alert function)

```
            TRIGGER_UNIFIED_ALERT ("Hazard_Localized", "Smart Vision
O1", HAZARD_INFO)
            BREAK
// Alert on first critical hazard detected per frame
        END IF
    END FOR

    WAIT_FOR_NEXT_FRAME () // Wait for the next frame processing cycle
END WHILE
END FUNCTION
```

SECTION 4: UNIFIED ALERT AND DATA FUSION (O3)

```
FUNCTION TRIGGER_UNIFIED_ALERT (ALERT_TYPE, ALERT_SOURCE,
OPTIONAL_HAZARD_INFO = NULL):
// CRITICAL: Ensure sub-second latency goal is met for intervention.
```

// 1. Local Intervention (Immediate Feedback to Worker)

```
ACTIVATE_LED (LED_LOCAL_ALERT, COLOR_RED)
LOG_LOCAL_EVENT ("Local Alert Triggered: " + ALERT_TYPE)
```

// 2. Secure Data Transmission (O3: Data Fusion Preparation)

```
GPS_LOCATION = GET_LOCATION(COMMS_SIM7670G)
// Obtain real-time coordinates

// Construct the Unified Alert Packet
ALERT_PACKET = {
    TYPE: ALERT_TYPE, TIME: NOW (),
    LOCATION: GPS_LOCATION,
    SOURCE: ALERT_SOURCE,
    WORKER_ID: GET_CURRENT_USER_ID (),
    APP_ID: GET_APP_ID (),
    HAZARD_DETAILS: OPTIONAL_HAZARD_INFO // Includes localization data
if O1 triggered it
}
```

// 3. Cloud Communication (Secure, Real-Time Upload via 4G/LTE)

```
SEND_SECURE_DATA (ALERT_PACKET, COMMS_SIM7670G, CLOUD_BACKEND_URL)
```

// 4. Dashboard Update (O3: Centralized Data Analytics)

```
// The CLOUD_BACKEND receives the packet, stores it.
// processes for fusion (Process Data for Dashboard), and PUSHES the
alert
// to the Safety Officer's Dashboard in real-time.
LOG_REMOTE_EVENT ("Alert Sent to Cloud for Centralized Monitoring.")

// De-activate local LED after a brief pulse to conserve power/avoid
distraction
WAIT (1.0)
DEACTIVATE_LED(LED_LOCAL_ALERT)
END FUNCTION
```

7.5 Prototype

The Halm Dashboard (O3) provides safety officers with a unified, real-time platform for monitoring worker safety, managing alerts, and conducting historical audits. It securely fuses multi-modal data streams (Vision O1, Sense O2, and GPS) into a single display, ensuring rapid, informed command-response actions to achieve proactive worker protection. This section details the 16 key user interface (UI) screens across four main functional areas.

7.5.1 User Authentication and Access Control (4 UIs)

This section manages user access, ensuring that only authorized safety officers can securely log in and gain situational awareness. Secure access begins with the Login Page Interface for credential validation. If access is blocked, the Forgot Password/Reset Workflow provides a quick, secure email-based process for regaining control. For immediate, pre-login assistance, the Help & Support Modal is available, providing emergency hotlines and troubleshooting steps.

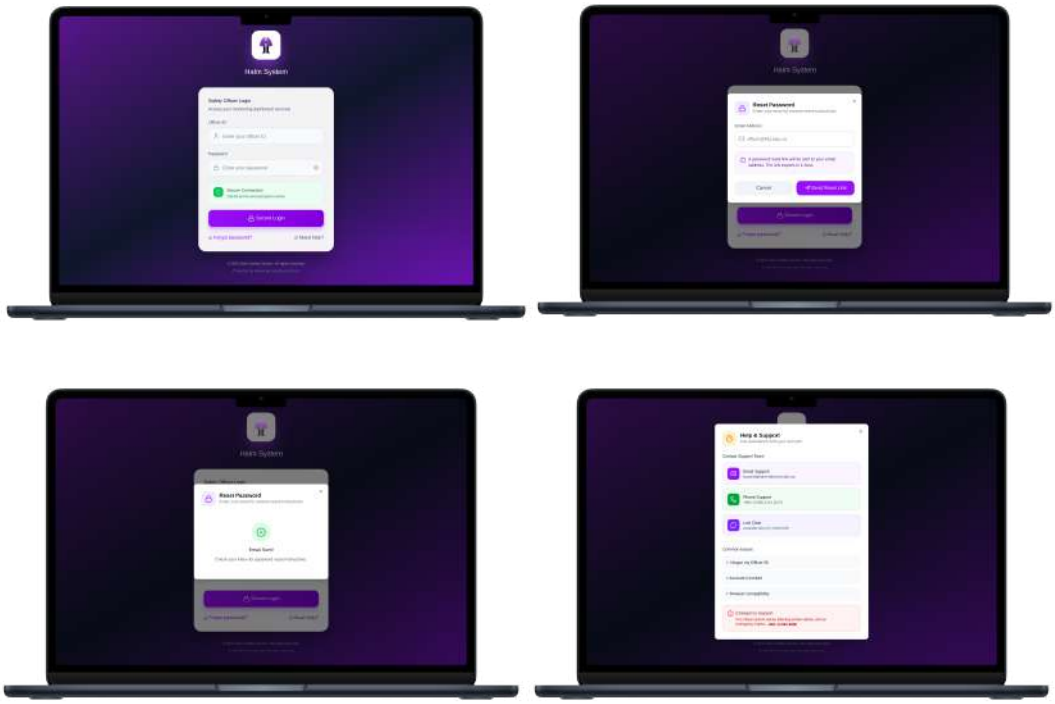

Figure 8: Halm Dashboard User Authentication Interfaces

7.5.2 Real-Time Dashboard and Situational Awareness (4 UIs)

These UIs provide core monitoring capabilities, integrating GPS and live alert data for an immediate, holistic view of the work environment. The core is the Live Situational Map View (O3), which displays the real-time GPS location and color-coded status of all active helmets. Critical information is presented through the Real-Time Alert Stream & List, a consolidated feed of incoming Smart Sense (Fall) and Smart Vision (Hazard) alerts. A Critical Alert Detail Modal automatically pops up upon an event, displaying incident specifics and immediate response guidance. The Worker Status and Telemetry Card present the latest individual data and the last known update time.

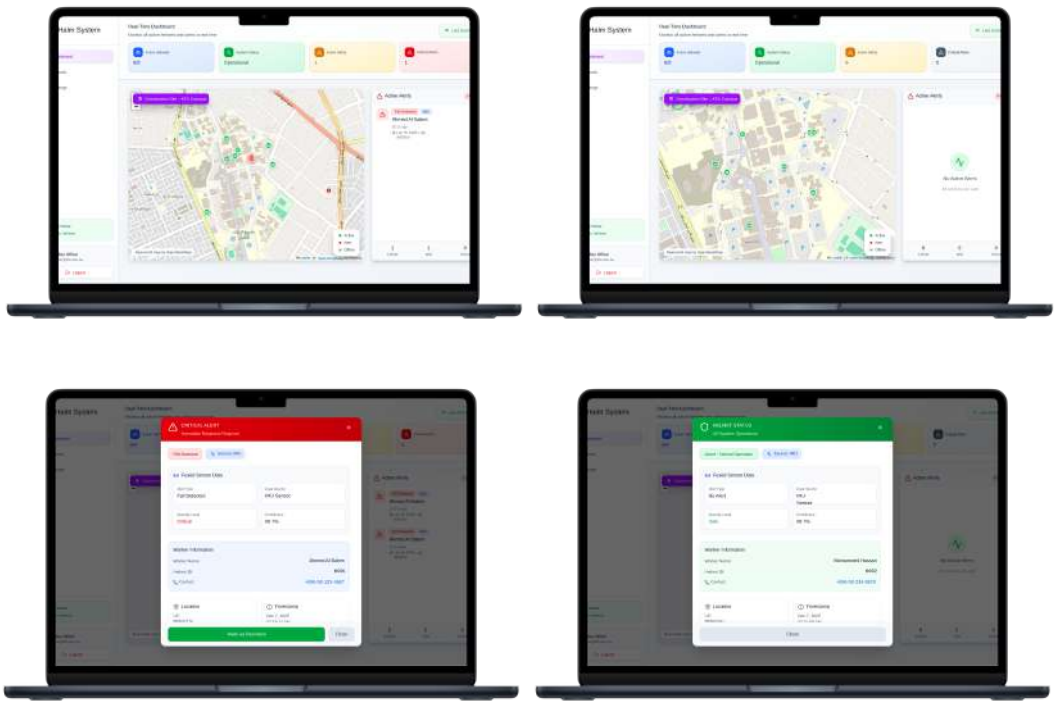

Figure 9: Halm Dashboard Real-Time Situational Awareness Interfaces

7.5.3 Historical Reports & Analytics (4 UIs)

This section provides tools for safety auditors to analyze past incidents, identify trends, and inform future mitigation strategies based on collected multi-modal data. For retrospective auditing, the platform uses the Historical Incident Log Viewer, a searchable table of all archived alerts. This data can be refined using the Date Range and Filter Selection interface to analyze specific periods or alert types. Key performance insights are presented through Safety Trend Visualization (Charts), which summarize metrics like incident frequency. The Export Report Functionality allows downloading comprehensive incident data for external record-keeping and compliance.

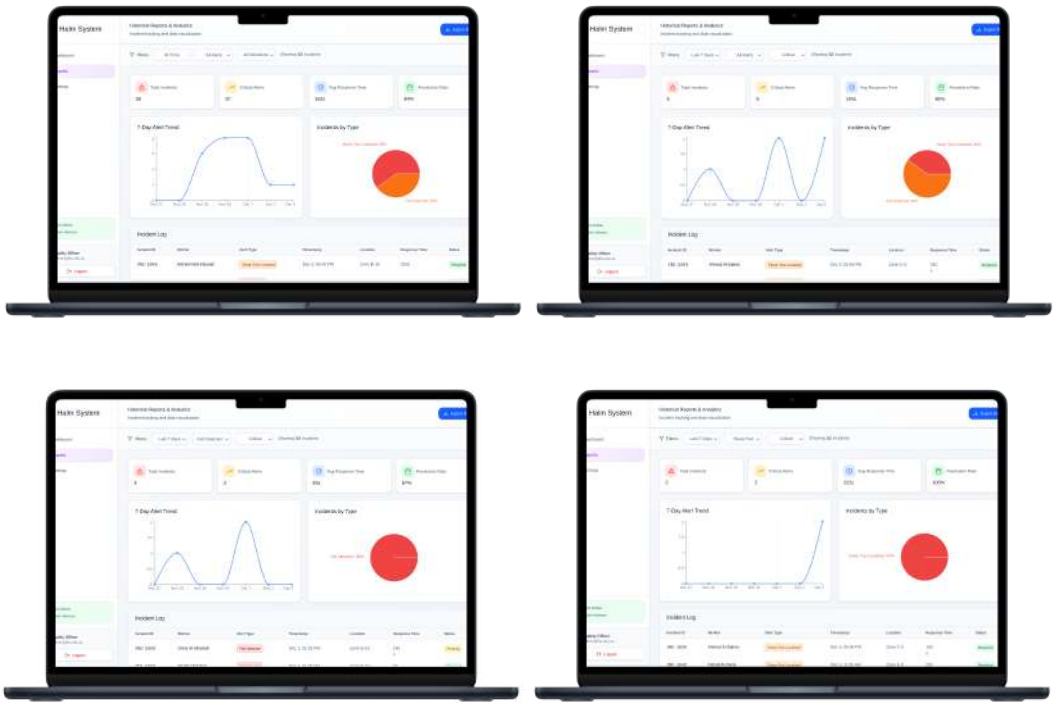

Figure 10: Halm Dashboard Historical Reports and Analytics Interfaces

7.5.4 Settings, User Management, and System Configuration (4 UIs)

These interfaces manage user accounts, system configuration, and provide administrative control over the platform. System administration begins with User Account and Profile Settings, allowing officers to control their personal credentials. Finally, the Security and Admin Controls screen offers security settings, including Two-Factor Authentication (2FA) and irreversible administrative actions, such as clearing alert history.

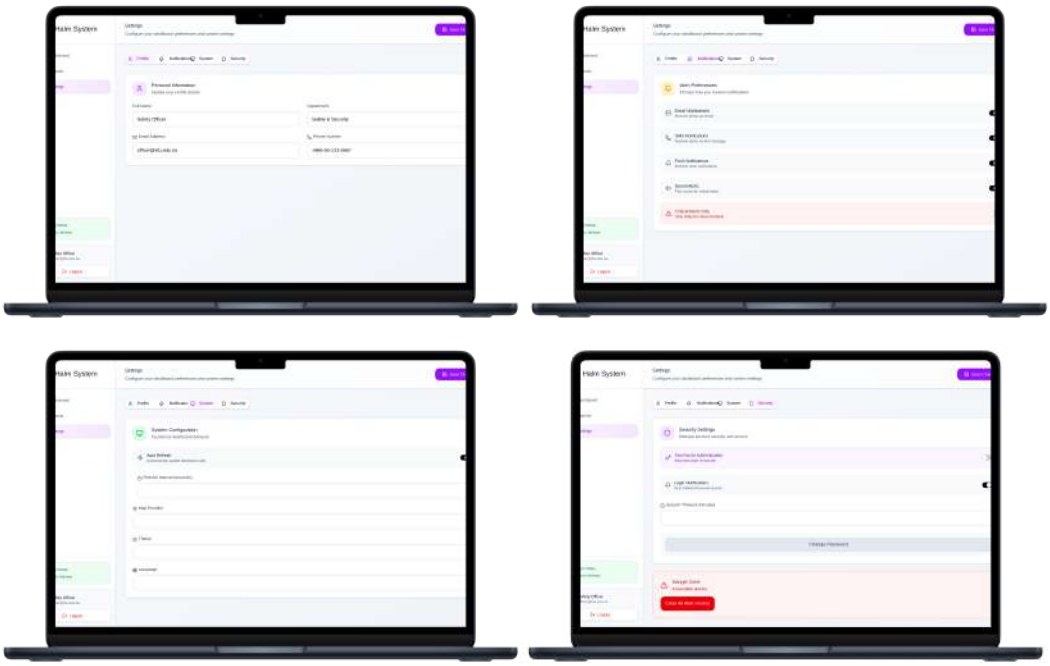

Figure 11: Halm Dashboard Administrative and Configuration Interfaces

7.6 Ethical & Legal Considerations

The implementation of the Halm must preemptively counter the major ethical, privacy, and legal issues associated with continuous worker monitoring. Halm is envisaged as a safety device rather than a surveillance mechanism; therefore, its architecture adheres strictly to the principles of data minimizing and transparency. All design decisions align with the Saudi Personal Data Protection Law (PDPL) [26], which mandates data minimization, clear purpose specification, informed consent, secure data handling, and destruction of data when no longer required (Articles 4-5, 10-13, 18-19, 23).

7.6.1 Privacy and Data Security by Design

The most dominant ethical issue is balancing the potential invasion of worker privacy with the maintenance of high safety standards. To mitigate the risk of continuous surveillance via the camera, GPS, and IMU data streams, Halm focuses on edge processing on the Raspberry Pi 5. High-frequency IMU measurements (O2) and raw video frames (O1) are handled locally in the helmet. The system also complies with data-minimization principles, transmitting data to the cloud only when a critical event (e.g., fall or hazard) is decisively detected, or when a non-essential, anonymized system diagnostic is required. This critical design choice guarantees that the cloud infrastructure only stores incident-related data needed to respond to an emergency, rather than recording the ongoing work history of the workers.

Legally, protecting this sensitive information is essential. To prevent interception during transmission, all communication between the helmet's T-SIM7670G module and the cloud backend must utilize End-to-End Encryption (E2EE). Furthermore, the Centralized Data Analytics Dashboard (O3) enforces strict role-based access control to ensure only authorized safety officers can access real-time data and historical reports.

7.6.2 Autonomy, Transparency, and AI Bias

The acceptance of the workers depends on the transparency of the system’s functionality. The employer needs a clear policy, supported by informed consent, stating that Halm's data is used solely for safety and emergency response purposes. This policy

must legally restrict the use of Halm's data for performance evaluations or other disciplinary measures to keep the workers at ease to avoid non-observance. The local feedback of the WS2812B at the RGB light alert that is received immediately on the spot is more reassuring and helps the system to become a personal safety helper and not a remote monitor. Finally, Halm must ensure fair safety among the entire staff by considering the possibility of AI bias. Both the YOLOv5-Nano (O1) model and the Random Forest (O2) model will have to be tested on a well-representative data source, including light and weather variations, body motions, and location variations. The reason why this measure is important is to ensure that accuracy of the detection is maintained and is consistent, hence creating fair protection for all workers irrespective of their work environment as well as personal traits.

8 System Implementation and Development

This chapter presents the transition of the Halm project from a conceptual architectural framework into a functional, deployed safety system. It examines the development process, the technical stack employed, and the engineering refinements introduced to bridge the gap between the initial design specifications and real-world operational requirements.

8.1 Implementation Progress Overview

This section summarizes the development progress across the system's three primary modules. It outlines the successful implementation of the YOLOv8-based Smart Vision Module for hazard detection, the IMU-driven Smart Sense Module for fall detection at the edge, and the construction of the dashboard for safety monitoring and alerts.

8.1.1 Smart Vision Module (O1) – Hazard Detection

The Smart Vision Module (O1) was implemented using an edge-optimized YOLOv8 model trained on construction-related datasets, including pits, sharp tools, and handheld tools to improve contextual hazard interpretation. The addition of the handheld tools class enables the system to distinguish between tools actively used by workers and tools left unattended, reducing false alerts, and was deployed on a Raspberry Pi for real-time edge inference and validation.

8.1.2 Smart Sense Module (O2) – Fall Detection

The Smart Sense Module has been successfully implemented and validated at the sensor level. The fall detection mechanism is based on the BNO085 inertial measurement unit (IMU), which provides real-time tri-axial acceleration data. The sensor was programmed through the Adafruit BNO08x I^2C interface, which allowed receiving stable acceleration indicators due to direct contact with the Raspberry Pi. The fall detection algorithm was performed using a threshold-based algorithm, which analyzed the magnitude of the acceleration vector calculated as the Euclidean norm of the three axes. This approach enabled continuous monitoring of acceleration changes and allowed the system to identify abnormal motion patterns associated with potential fall events.

8.1.3 Dashboard and Web System (O3)

The centralized dashboard system has reached a high level of implementation. The web application has been built to provide real-time visualization of detected hazards and system-generated alerts. The user interface design and general layout have been applied to ensure clarity and usability for safety monitoring purposes. Backend API structures and data handling mechanisms have been prepared to support future integration with the edge processing unit.

8.2 Tools and Techniques Utilization

8.2.1 Hardware Components

This section presents the primary hardware components integrated into Halm system and summarizes their functional roles within the edge-based safety architecture.

Table 10 Hardware Components

Hardware Component	Description and Functional Role
Raspberry Pi 5	Used as the central edge processing unit. It runs the computer vision inference model, executes fall detection logic using IMU data, manages sensor communication, and coordinates the local alert mechanism.
BNO085	Provides tri-axial accelerometer and gyroscope data required for motion monitoring and fall detection. Supports high-frequency sensing and communicates via I²C with the Raspberry Pi. Selected for compact size and suitability for wearable systems.
Raspberry Pi Camera	Captures real-time video frames processed by the AI model for hazard detection. Enables continuous environmental monitoring and connects via the CSI interface for low-latency image acquisition.
T-SIM7670G	Supports LTE connectivity and GPS positioning. Enables remote alert transmission and location tracking within the system architecture.
WS2812B	Functions as a local visual alert mechanism embedded in the helmet, providing immediate on-device feedback during hazard detection events.

8.2.2 Artificial Intelligence and Computer Vision

The computer vision system is based on YOLOv8 Nano, selected for its lightweight architecture and suitability for embedded edge deployment. The model was trained to

detect construction-related hazards, including ground-level sharp tools, handheld sharp tools, and potholes. Training was conducted using curated datasets, and validation procedures were performed to ensure detection stability before deployment. The finalized model was then prepared for embedded inference and deployed on a Raspberry Pi 5 for real-time hazard detection under controlled conditions.

8.2.3 Programming Languages

Python served as the primary programming language across all system layers of the Halm system. It was used to obtain sensor data in the inertial measurement unit (IMU) module, real-time processes of acceleration signals to detect falls, execution of the computer vision, and running real-time AI inference using the deployed YOLOv8 model.

8.2.4 Backend and Communication Layer

The backend infrastructure was developed using Flask, which provides lightweight API endpoints to manage incoming detection events and system-generated alerts. The backend architecture was designed to support future integration between the edge device and the web dashboard, establishing a scalable framework for structured data transmission in subsequent development stages.

8.2.5 Frontend and Interface Development

The following table outlines the specific design tools and web technologies used to develop the dashboard's user interface.

Table 11 Frontend and Interface Development

Technology	Usage in the system
React	Developed the dynamic and responsive dashboard interface, enabling a modular UI structure and efficient state management for real-time alert rendering.
HTML	Used to structure the dashboard layout and define content organization.
CSS	Implemented to manage styling, visual hierarchy, and interface presentation.
JavaScript	Used to manage interactive behaviors within the dashboard interface
Figma	Used during the early design phase for UI/UX prototyping, wireframing, and structured interface planning before implementation.

8.3 Design-Implementation Consistency and Optimization

8.3.1 System Architecture and Stack Alignment

The implementation adheres to the decoupled architecture defined during the design phase. The backend is developed using Flask and is responsible for computationally intensive operations, including YOLO-based inference, alert processing, and database management. The frontend is implemented using React and Vite, enabling the development of a responsive, high-performance administrative dashboard.

Database interaction is managed via SQLAlchemy, which operates the conceptual schema into a streamlined three-table SQLite database (Users, Workers, and Alerts). This refined architecture, discussed further in Section 8.3.1, ensures high-speed data integrity and structural consistency. Complementing this, Flask-CORS is integrated to bridge the decoupled components, enabling secure cross-origin communication and ensuring seamless, real-time alert transmission between the backend API and the frontend dashboard.

8.3.2 Hardware-Software Integration

To maintain consistency between the proposed design and the implemented system, initial bench testing was conducted to verify the functional integration between the Halm software modules and the hardware components. This phase confirmed that the core system requirements can operate in real time on the edge device, the Raspberry Pi 5.

- **Smart Vision Module and Camera Orientation:**

The hazard detection model based on YOLOv8 was successfully executed on the edge device using live input from the camera module. At this stage, the camera is designed to be mounted with an approximate 45° downward tilt. This orientation prioritizes the visibility of ground-level hazards, including potholes and sharp tools, while maintaining the worker's natural field of view. Additionally, the downward viewing angle reduces the presence of irrelevant objects in the frame, which helps minimize unnecessary detections. Final verification and validation of the camera position will be completed during Milestone 4, when the physical helmet assembly takes place.

- **Smart Sense (Kinematic Logic) Verification:**

In parallel with the vision subsystem, the IMU-based fall detection mechanism was tested using the BNO085 sensor. Bench testing confirmed that the sampling-based dual-threshold logic reliably detects abnormal motion patterns while maintaining minimal CPU overhead. This verifies the suitability of the sensing hardware and detection logic for real-time operation on the edge device.

- **Integration Status:**

Although the computer vision and kinematic sensing subsystems have been independently validated for hardware compatibility, the complete end-to-end system integration, which links hardware-generated alerts to the web-based dashboard, will be completed during the final system integration stage. Detailed implementation data, wiring configurations, and experimental evaluation are presented in Chapter 9.

8.3.3 Unified Multi-Class Detection Pipeline

In alignment with the original resource-efficiency objective, the system employs a single unified multi-class object detection model rather than multiple parallel models. This approach enables simultaneous detection of Sharp Tools, Handheld Tools, and Potholes while minimizing memory and CPU overhead on edge hardware. This architectural decision ensures real-time performance on the Raspberry Pi 5 without compromising detection coverage. To support this unified pipeline, the training phase involved a strategic consolidation of specialized datasets to ensure high accuracy across all detection classes.

8.3.4 Dataset Integration and Refinement

To facilitate the transition from conceptual design to functional implementation, the Halm data requirements were re-evaluated. This phase involved the strategic selection of previously identified datasets alongside the integration of new sources to optimize model accuracy.

- **Small-Size Construction Tools Dataset [23]:** As proposed in Chapter 4, this remains the primary source for identifying sharp hazards on the ground. Its high visual diversity was essential for the vision model's baseline training.

- **Annotated Potholes Dataset [24]:** In addition to the Kaggle-sourced dataset mentioned in Chapter 4 [24], this dataset was combined with a high-resolution pothole dataset from Roboflow [28] to enhance sample diversity and improve robust detection across varying site conditions.

- **Handheld Sharp Tools Dataset [29]:** A new specialized dataset was integrated during this phase. This was necessary to train the model to distinguish between active hazards and safe tool usage, a critical step in reducing false alarms.

- **IMU Data Revision [25]:** Although an IMU dataset was initially proposed in Chapter 4, the implementation phase moved toward a deterministic signal-processing approach. Consequently, the raw dataset was not used for model training, as the system now relies on real-time threshold-based sampling to ensure immediate response.

8.4 Justification for Design Modification

8.4.1 Transition from YOLOv5 to YOLOv8 Small (YOLOv8s)

The initial design proposed the use of YOLOv5. During implementation, the system transitioned to YOLOv8 Small (YOLOv8s) to leverage architectural advancements.

- **Justification:**
  - **Architectural Efficiency:** YOLOv8 introduces an anchor-free detection, which improves localization flexibility and reduces dependency on predefined anchor boxes. This architectural improvement makes it more suitable for detecting small and irregularly shaped objects such as sharp construction tools.

  - **Safety-Critical Performance:** A comparative evaluation between YOLOv8 Nano and YOLOv8 Small demonstrated that YOLOv8s achieved superior performance metrics (mAP50 = 0.99) and (Recall = 0.97). Since Recall measures the ability to detect actual hazards, prioritizing higher recall minimizes False Negatives.

- **Risk Mitigation:** In a construction safety context, failing to detect a hazard presents significantly greater risk than marginal gains in inference speed. The improvement in detection reliability justified selecting YOLOv8s over the Nano variant.

8.4.2 IMU Logic Refinement: From Random Forest to Average Sampling

The original design proposed a Random Forest classifier to process data from the BNO085 IMU sensor for fall detection. During implementation, this approach was replaced with a threshold-based Average Sampling method.

- **Justification:**
  - **Logic Simplification:** Preliminary testing demonstrated that deterministic average sampling provided sufficient sensitivity for fall detection without the computational overhead associated with a probabilistic machine learning classifier.
  - **Resource Allocation:** Eliminating the secondary machine learning classifier reduced CPU utilization on the Raspberry Pi 5, reserving power for the more demanding computer vision pipeline.
  - **Response Speed:** The signal-processing approach ensures immediate alert triggering, which is vital for time-sensitive emergency responses in the event of a worker falling.

8.4.3 Technical Stack Optimization (React + Vite and Flask-CORS)

While earlier chapters identified core technologies, the implementation phase necessitated the adoption of additional tools to bridge the gap between design and functionality. These enhancements represent a maturation of the technical stack to support a real-time, scalable environment.

- **Justification:**
  - **Performance Enhancement:** The transition from a standard React setup to Vite significantly improved frontend build times and dashboard responsiveness, ensuring a seamless user experience during real-time monitoring.

- **Integration Security:** The implementation of Flask-CORS was essential to manage cross-origin requests, allowing the decoupled frontend and backend components to communicate securely and without latency.

- **Data Integrity:** The use of SQLAlchemy as a robust Object-Relational Mapper (ORM) ensured a reliable mapping between the Python backend and the SQLite database, facilitating consistent data handling and integrity across the system.

8.4.4 **Database** Evolution (Revised ER Diagram and Schema)

The database schema evolved from the conceptual Milestone 2 design into a refined three-table relational structure consisting of User, Worker, and Alert.

- **Justification:**
  - **Improved Response Time:** Reducing the schema from six tables to three minimizing the data processing layers. This ensures that hazard detections are logged and displayed on the dashboard with near-zero latency.

  - **Simplified Integration:** A streamlined database makes the communication between the Raspberry Pi's detection script and the Flask backend more efficient, reducing the complexity of the API calls. As shown below in Figure 12, this is the revised version of the ER diagram, reflecting the streamlined schema for Users, Workers, and Alerts.

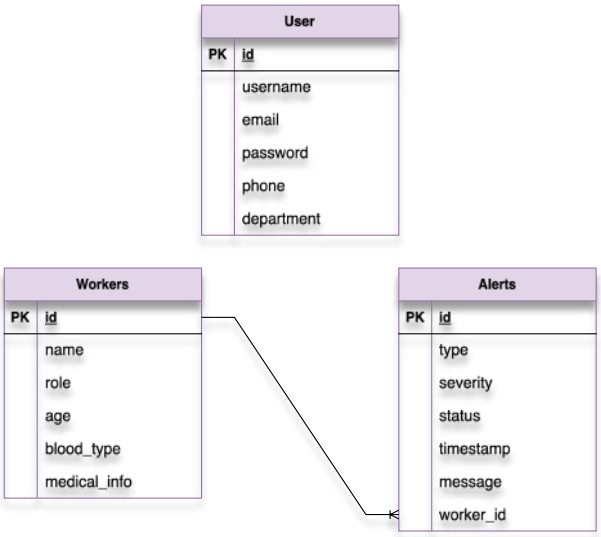

Figure 12: Refined Database Schema and Entity Relationships

8.5 Compliance with Professional Practices and Standards

The implementation of the Halm system strictly adhered to established software engineering and ethical standards within the project's practical scope. From an engineering perspective, we utilized an Agile methodology, enabling iterative testing and systematic integration between the Raspberry Pi 5 edge hardware and the Flask-based web dashboard. Code modularity and standard relational database practices (SQLite) were applied to ensure system stability and reliable data management.

Ethically, the project applied "Privacy by Design" principles to align with the Saudi Personal Data Protection Law (PDPL), as initially established in Section 7.6. By processing video streams locally on the edge via the Raspberry Pi, the system actively protects worker privacy; no continuous surveillance footage is transmitted. Instead, only specific hazard alerts, timestamps, and metadata are sent to the database. Furthermore, professional AI practices were observed by training the YOLOv8 models on balanced datasets to mitigate algorithmic bias. System security was practically enforced through robust user authentication protocols, ensuring that sensitive personnel health records on the dashboard remain accessible solely to authorized safety officers.

9 System Outcomes and Results

This chapter validates the Halm system's performance by documenting its transition from design to implementation. It focuses on three key areas: benchmarking the hazard detection models, evaluating the functionality of the monitoring dashboard, and deploying on Raspberry Pi 5 to ensure real-time safety at the edge.

9.1 Technical Model Evaluation and Selection

The reliability of the Halm system hinges on the precision of its computer vision engine. We conducted a comparative benchmark between YOLOv8 Nano and YOLOv8 Small to identify the optimal balance between speed and accuracy for construction safety. As shown in Table 12, the YOLOv8 Small model demonstrated a significant advantage in all safety-critical metrics. With a mAP@50 of 99.3% and a Recall of 0.975, it ensures near-perfect hazard localization while minimizing missed risks, a factor where the Nano version fell short. Despite the higher computational cost (28.6 GFLOPs), the Raspberry Pi 5 successfully handled the inference load, maintaining real-time responsiveness. Consequently, YOLOv8 Small was selected as our core model, prioritizing worker safety and high-fidelity detection over the marginal speed gains of the Nano variant.

Table 12: Technical Benchmarking (Nano vs. Small)

Metric	YOLOv8 Nano (n)	YOLOv8 Small (s)	Technical Significance
Precision (P)	0.854	0.985	Measures the accuracy of detections.
Recall (R)	0.812	0.975	Measures the ability to find all hazards.
mAP@50	0.881	0.993	The mean average precision at 50% intersection over union.
mAP@50-95	0.642	0.882	Represents overall localization accuracy on different scales.
Complexity (FLOPs)	8.7B	28.6B	Reflects the depth of the neural network layers.
Inference Time	~0.3ms	~0.5ms	Real-time processing speed per frame.

9.1.1 Visual Evaluation of Model Performance

To further illustrate the comparative analysis, the following figures present the visual performance metrics obtained during the training and validation phases. These graphs, including precision, recall, and mean average precision (mAP) trends, visually substantiate the superior stability and accuracy of the selected YOLOv8 Small model over the Nano variant.

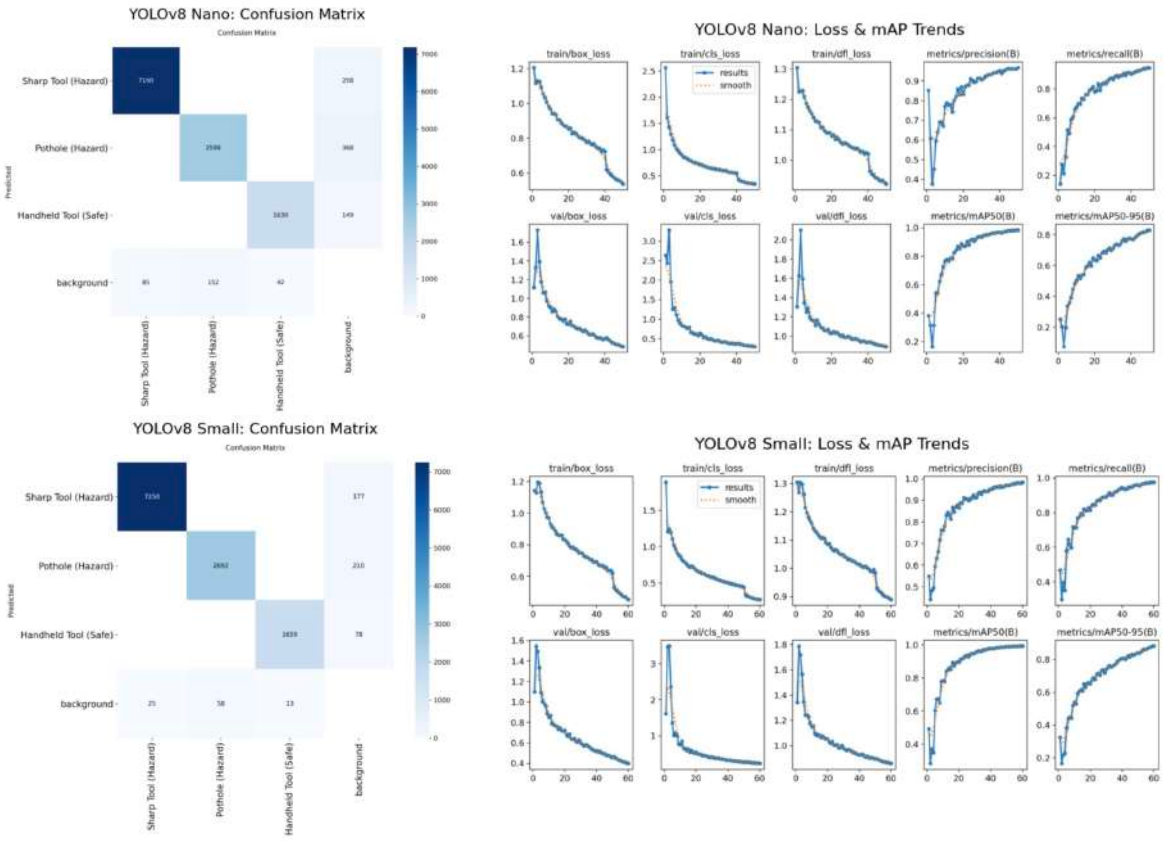

Figure 13: Visual Evaluation of Model Performance

9.2 Dataset Overview and Experimental Setup

The development of the Halm hazard detection system required a highly curated, multi-sourced dataset to ensure robust performance in dynamic construction environments. Because the system is designed to run on a Raspberry Pi using a unified YOLOv8 edge-computing model, the training data needed to encompass high visual diversity while remaining balanced. The final dataset comprises 6,409 images, yielding 11,697 labeled

bounding boxes across three distinct classes: Sharp Tools (Hazard), Potholes/Pits (Hazard), and Handheld Tools (Safe).

9.2.1 Sharp Tools on the Ground (Class 0)

The primary hazard targeted by the Halm helmet is abandoned sharp tools on the construction floor, which pose significant tripping and injury risks. The dataset for this class was sourced from DatasetNinja (Small-size Construction Tools) [23]. It is the largest component of our training data (4,249 images) and provides exceptional visual diversity, capturing various tools under different lighting conditions and background textures.

9.2.2 Combining Multiple Pothole / Pit Datasets (Class 1)

Detecting structural openings, pits, and uneven ground is vital for preventing fall-related accidents. To train this class, we strategically combined the Annotated Potholes dataset [24] with a High-Resolution Potholes dataset [28] to create a unified pool of 1,272 images. The rationale behind combining two distinct sources was twofold: first, to significantly increase the overall sample size for this specific hazard, and second, to introduce a wider variety of ground-level structural anomalies. A single dataset would risk overfitting the model to specific types of concrete or lighting. Merging them guarantees that the YOLOv8 model generalizes better to unpredictable, real-world construction terrains.

9.2.3 Handheld Sharp Tools for False Positive Mitigation (Class 2)

A critical design decision in the Halm system was the integration of a "Handheld Sharp Tools" dataset [29]. In a busy construction environment, workers are continuously holding and using tools correctly. If the model were trained solely to identify the presence of a tool, it would trigger continuous, erroneous alarms whenever a worker simply held a piece of equipment. By introducing an explicit "safe" class consisting of 888 images of tools actively held in human hands, the model learns the contextual difference between an abandoned tool (hazard) and a tool in use (non-hazard). This mitigates false positives and ensures the helmet's alarms are only triggered by genuine ground-level threats.

9.2.4 Experimental Setup and Training Parameters

The evaluation and training of the Halm model were conducted within a strictly controlled experimental environment using the YOLOv8 Small (YOLOv8s) architecture. For the training process, the unified dataset of 6,409 images was partitioned using a standard 70%, 20%, and 10% split for training, validation, and testing, respectively, ensuring unbiased evaluation on unseen data. Additionally, all input images were normalized to a resolution of 640x640 pixels. This specific resolution serves as the optimal standard for YOLOv8, striking the necessary balance between preserving the minute details of small ground hazards and maintaining high-speed inference capabilities.

The training phase was configured for a total of 60 epochs. To optimize the training cycle and proactively prevent overfitting, an early stopping mechanism was implemented with a patience parameter of 10. This allowed the training process to halt automatically if no validation improvement was observed over 10 consecutive epochs. Furthermore, to maximize computational efficiency, the training utilized an auto-batching configuration (batch=-1). This hardware optimization dynamically calculated the maximum allowable batch size, guaranteeing full utilization of the assigned Google Colab GPU memory without causing any memory overflow during execution.

Table 13: Unified Dataset Composition and Experimental Setup Summary

Dataset Category	Primary Source(s)	Merged Images	Labeled Objects	Purpose / System Role
Sharp Tools (Hazard)	DatasetNinja (Small-size Tools) [23]	4,249	7,275	Core hazard detection for tools abandoned on the ground.
Potholes / Pits (Hazard)	Roboflow & Kaggle (Chitholian) [28],[24]	1,272	2,750	Combined to increase sample size and visual variety of structural fall hazards.
Handheld Tools (Safe)	Roboflow (jee-cmmsh) [29]	888	1,672	False Positive Mitigation: Teaches the model to recognize safe, correct tool usage.
Overall Dataset & Setup	Consolidated Unified Model	6,409	11,697	Model: YOLOv8s Epochs: 60 Resolution: 640x640

9.3 Dashboard Interface Implementation

The Halm centralized dashboard was developed to provide a seamless and highly intuitive user experience, ensuring that critical safety data is easily accessible to administrators. The implementation successfully translates the initial prototype designs into functional, high-fidelity web interfaces, prioritizing both usability and system security.

9.3.1 User Authentication and Access Control

The User Authentication and Access Control module serves as the secure gateway to the Halm dashboard, ensuring exclusive access for authorized safety officers. Advancing beyond the static UI prototypes, this fully implemented module is now actively integrated with the system's SQLite database via the Flask backend. This crucial transition elevates the interface from a visual concept into a fully functional portal capable of real-time credential validation. By coupling robust backend data security with the clean, professional design and dynamic visual feedback of the original prototype, the implemented module successfully delivers both rigorous access control and a seamless user experience.

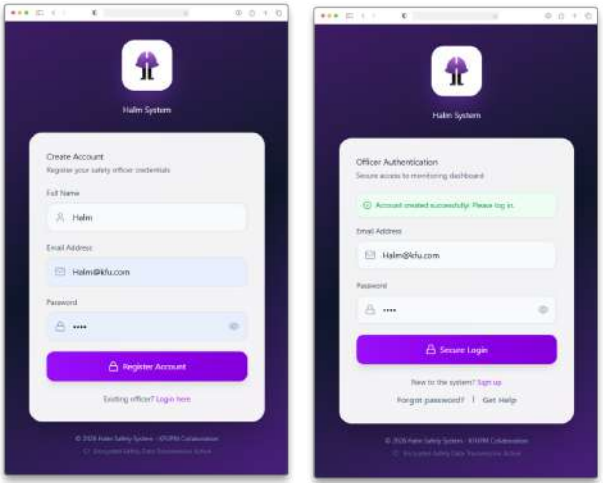

Figure 14: Halm Dashboard User Authentication Interfaces

9.3.2 Real-Time Dashboard and Situational Awareness

Structured to prioritize critical data visibility, the Real-Time Dashboard provides an immediate, top-down overview of the active work environment. At the uppermost section, dynamic KPI cards instantly quantify active personnel, specific hazard occurrences, and total resolved incidents. Moving downwards, the interface employs a

dual-view monitoring layout. On the left, a Live Site GPS Feed plots personnel locations using simulated coordinates to demonstrate spatial tracking capabilities pending final sensor deployment. On the right, the fully functional Safety Live Feed processes real-time camera streams to display instantaneous AI hazard detections. This feed actively extracts authentic, dynamic variables such as the worker's specific identity, exact timestamps, and AI confidence levels. The workflow concludes with an interactive Critical Alert Modal, enabling administrators to review granular incident details and securely update resolution statuses within the centralized database.

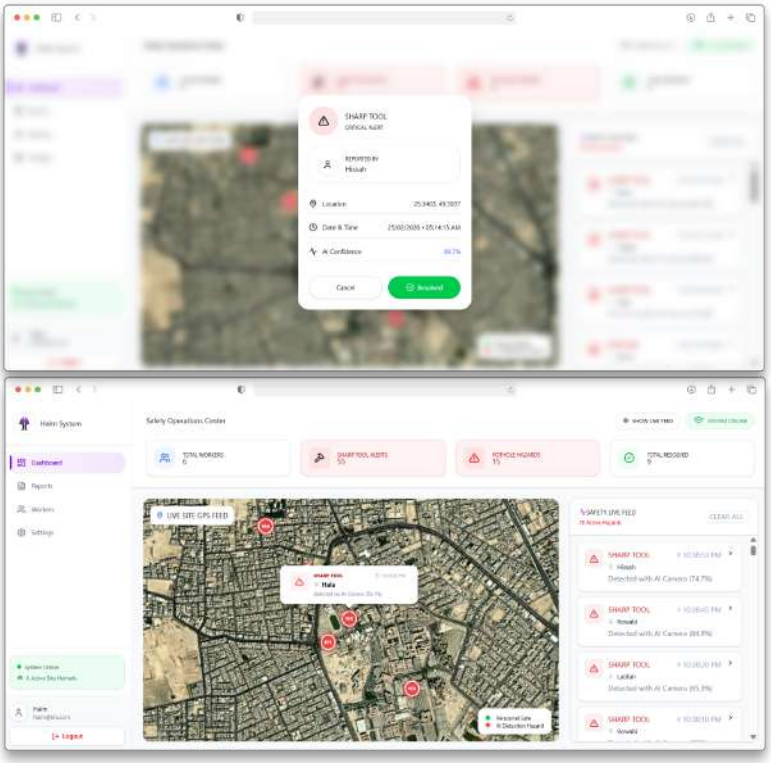

Figure 15: Halm Dashboard Real-Time Situational Awareness Interfaces

9.3.3 Smart Vision and Live Hazard Localization Implementation

Serving as the visual core of the Halm framework, this module integrates the edge-optimized YOLOv8 Small model to deliver real-time spatial hazard localization. While current dashboard visualizations represent a controlled testing phase, the final deployment will utilize the helmet-mounted Raspberry Pi Camera V2 for complete edge processing. The implemented "Live AI Vision" actively processes live video streams, autonomously drawing bounding boxes to distinguish critical threats such as a "Sharp Tool" or "Pothole" from non-threatening objects like a "Handheld Tool (Safe)." Crucially, this vision engine

is directly linked to the dashboard's notification system; upon identifying a risk, it instantly transmits an automated alert to the Safety Live Feed, embedding the exact AI confidence level values alongside the incident details. This seamless synchronization between visual detection and active alerting validates the system's capability for hazard management.

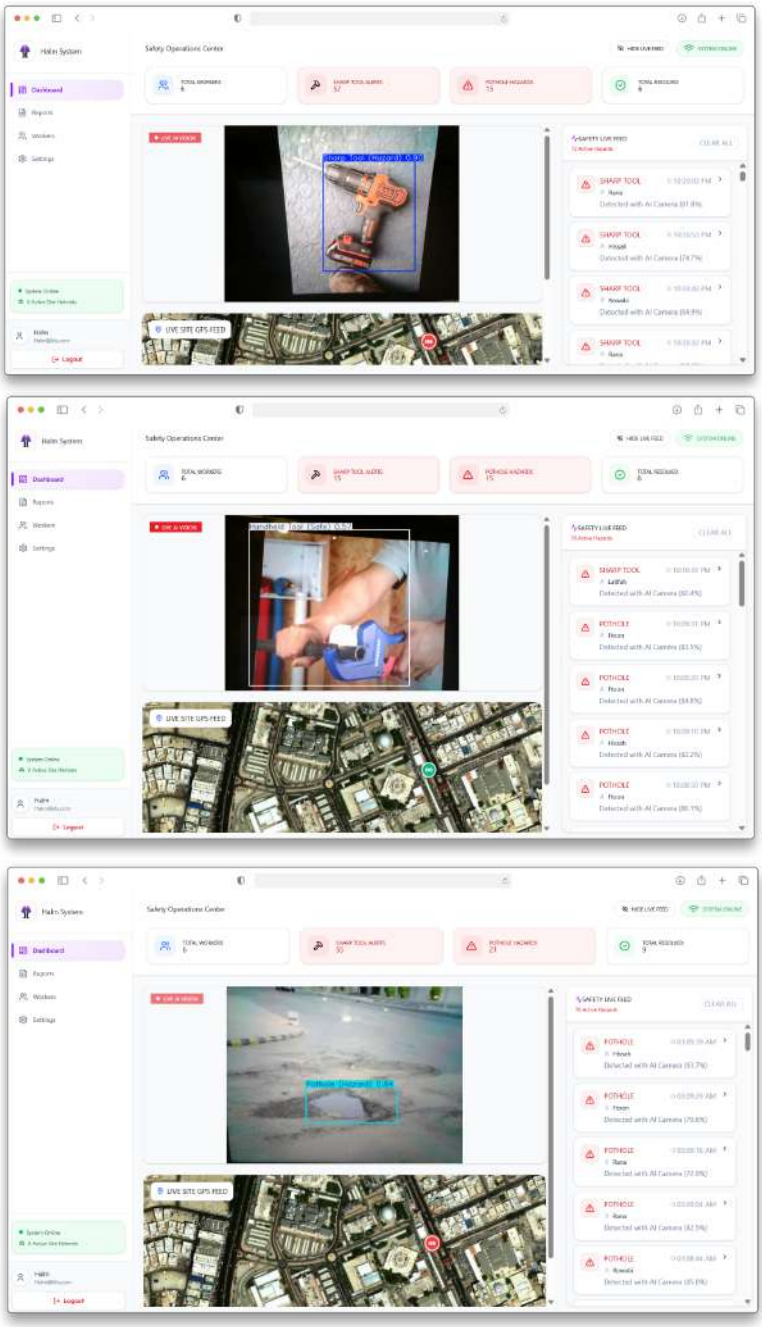

Figure 16: Smart Vision and Live Hazard Localization Implementation

9.3.4 Historical Reports and Analytics Implementation

Developed to support long-term safety auditing and post-incident analysis, this module transforms raw database records into actionable visual insights. The interface is structurally organized to present data hierarchically, beginning at the top with dynamic KPI cards that immediately quantify critical metrics such as total active hazards, resolved issues, and specific threat categories. Beneath this high-level summary, the dashboard incorporates interactive data visualizations, specifically a daily incident bar graph and a hazard distribution chart, enabling safety officers to quickly identify recurring operational trends. The lower section, "Detailed Activity Log," is a comprehensive table directly querying the SQLite database to display records, including worker identities, AI confidence scores, and precise incident timestamps. To facilitate external auditing and organizational compliance, the module concludes with a seamless, one-click CSV export function, allowing administrators to securely extract and share archived safety data.

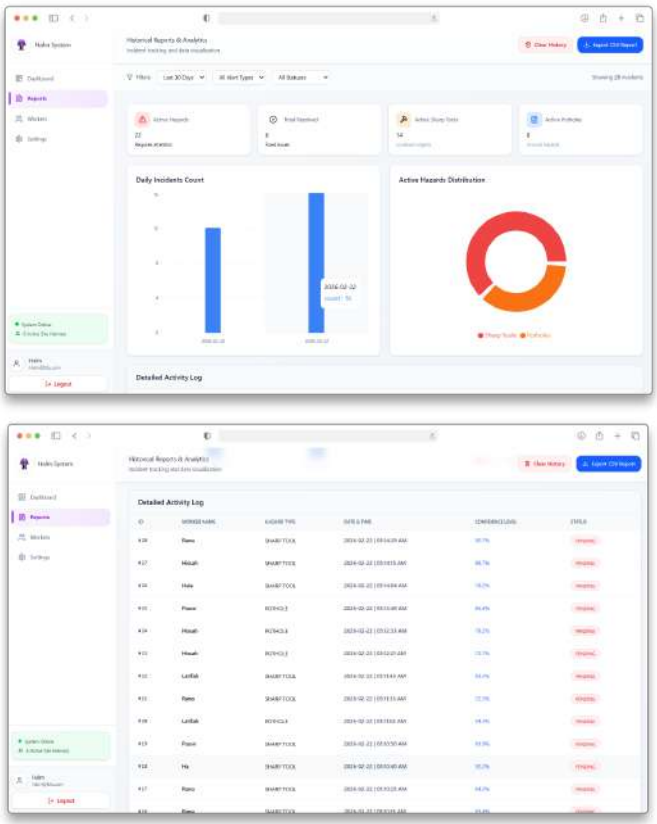

Figure 17: Historical Reports and Analytics Implementation

9.3.5 Personnel Management and Safety Profiling Implementation

Serving as the administrative of the Halm platform, the Workers Management module provides a secure environment for handling vital personnel data. The interface organizes the workforce into an intuitive grid of profile cards, strategically highlighting crucial emergency metrics such as blood type and medical conditions alongside standard demographics. For seamless onboarding, an interactive modal allows safety officers to efficiently input new worker credentials. Crucially, all additions, modifications, and deletions instantly synchronize with the central SQLite database, ensuring that administrators always rely on accurate, health profiles during emergency responses.

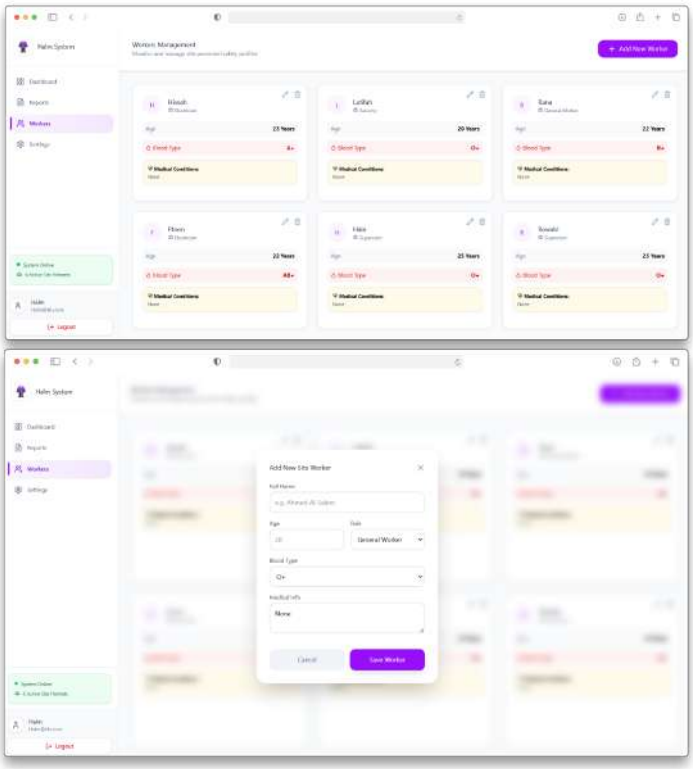

Figure 18: Personnel Management and Safety Profiling Implementation

9.3.6 Administrative Settings and System Configuration Implementation

Concluding the dashboard framework, the Settings module provides safety officers with comprehensive control over their operational environment and account security. The interface is systematically divided into four intuitive tabs. The "Profile" and "Notifications" sections allow administrators to securely update personal credentials and tailor multi-channel alert preferences such as SMS, email, and sound cues, ensuring customized and

rapid notification of site hazards. The "System" tab offers workspace personalization, enabling users to adjust map engines and visual themes. Crucially, the "Security" tab enforces robust account protection by integrating Two-Factor Authentication (2FA) capabilities alongside a strictly controlled "Danger Zone" for irreversible database wipes. All configuration adjustments are instantly synchronized with the Flask backend.

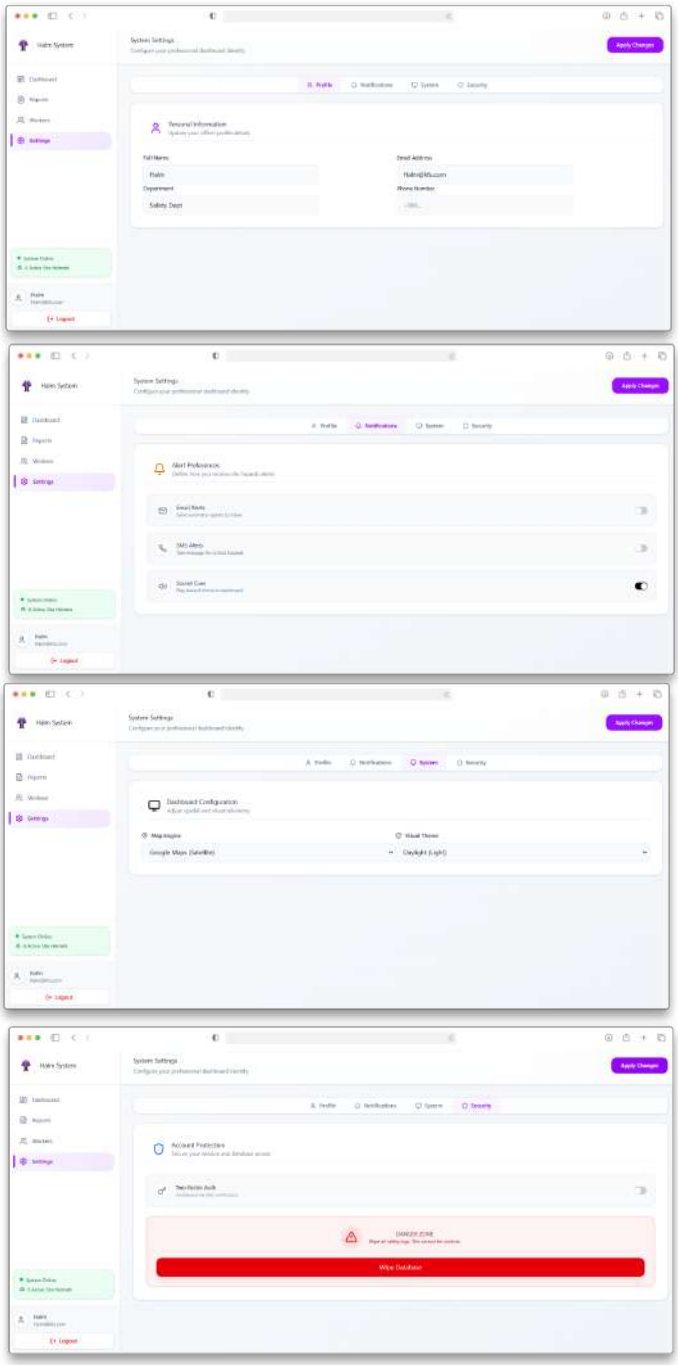

Figure 19: Administrative Settings and System Configuration Implementation

9.4 Hardware Integration and Edge Processing Implementation

Transitioning to physical deployment, the Raspberry Pi 5 was configured as the central edge computing unit. The hardware setup involved securely interfacing the Raspberry Pi Camera V2 via the CSI port and preparing the local environment by installing essential dependencies, including Python, OpenCV... The trained YOLOv8 Small model was then transferred directly to the device, enabling autonomous, offline inference. Live hardware testing validated the system's real-time edge processing capabilities. The model demonstrated precise contextual awareness, successfully detecting ground hazards like "Pits." Crucially, it accurately distinguished between an unattended hammer, classifying it as a "sharp_tool" hazard (0.90 confidence), and the same object was correctly recognized as a safe "Handheld_tool" (0.82 confidence) when actively grasped by a worker.

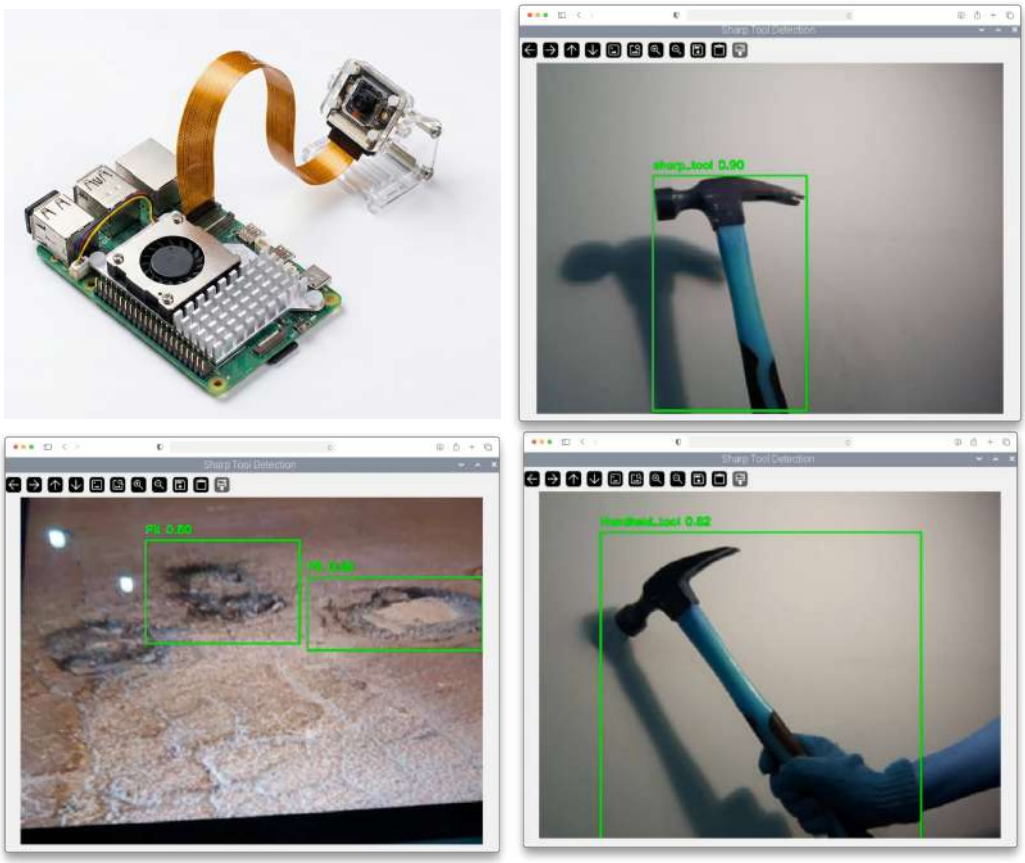

Figure 20: Administrative Settings and System Configuration Implementation

9.5 Kinematic Detection and Smart Sense Implementation

The fall detection system is one of the key safety elements of the Halm project, which is designed to provide real-time tracking and instant notification in the event of an occupational incident. The implementation process will be divided into hardware integration, environmental configuration, and algorithmic development.

9.5.1 Installation and Hardware Integration.

The sensor is attached to the Raspberry Pi and Pin 3.3 V power (Pin 1), Ground (Pin 9), SDA (Pin 3), and SCL (Pin 5). As shown in the wiring diagram (Figure 19), the four-wire design ensures reliable communication and minimal power consumption, both of which are important in a helmet application. The Raspberry Pi was prepared by enabling the I2C interface using the "raspi-config" command. Subsequently, I2C-tools were installed to verify the sensor's hardware address (0x4B) using the "i2cdetect" utility. Finally, the "adafruit-circuitpython-bno08x" library was utilized to facilitate high-level data acquisition from the IMU.

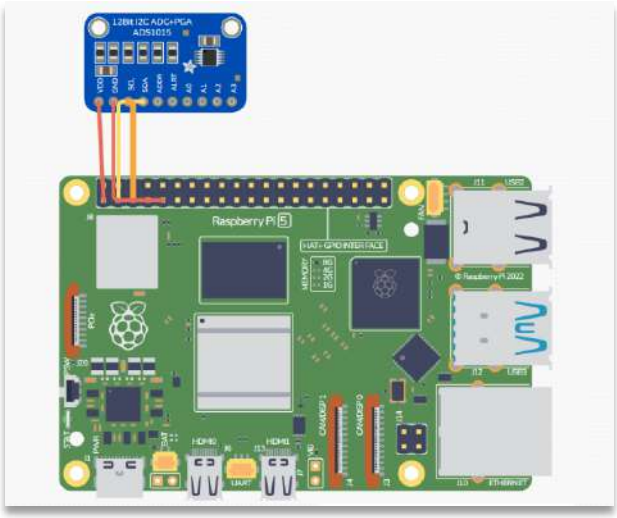

Figure 21: Wiring diagram of the BNO085 IMU sensor connected to the Raspberry Pi GPIO.

9.5.2 Fall Detection Algorithm

The fall-detection algorithm uses a dual-threshold validation model in order to differentiate normal motion and significant falls. The acceleration can be calculated through a vectorial magnitude of acceleration (A_{mag}) by the following formula:

$$A_{mag} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

The algorithm works in three consecutive steps:

1. **Free-fall detection:** whereby a system continuously checks the condition of falling weightless (A_{mag} falling below 4.0 ms^2) at least once during a 100 ms interval (validity period).
2. **Validation window**: in which a one-second window is opened to detect the next impact.
3. **Impact detection:** in which a critical alert has been triggered once the impact force is more than 45.0 ms^2 (or 4.5 g). This design reduces the number of false positives caused by the small perturbation or deliberate positioning of the helmet.

9.5.3 Results and Outcomes: Experiment.

The assessment stage determined that the Halm system is dependable to detect high-impact falls without generating activity during normal activity. The monitoring script is then run to initialize the I2C bus, and real-time sampling is then started at approximately 100 Hz. Figure 20 shows the output logs, which validate that the system recorded a fall event that was simulated successfully. The console output shows the shift of a state of a potential fall to the issuance of a critical alert at the point of going beyond the impact threshold. The measurement of impact force in this specific test was 57.17 ms^2, and this was recorded, which immediately triggered the notification protocol. A five-second cooling-off period after an alert is used to avoid saturation of the emergency network with messages on the same incident by the system. This evidence confirms that the novel BNO085 sensor, along with the Python-based detection logic, is a reliable foundation for worker protection.

```
--- Halm System: Monitoring Active ---
Potential fall, Mag: 2.16 - Monitoring for impact...
Potential fall, Mag: 3.16 - Monitoring for impact...

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
CRITICAL ALERT: IMPACT DETECTED
Impact Force: 51.12 m/s^2
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

Figure 22: Console output showing successful fall detection and impact force reporting

10 Challenges and Resolutions

The Halm system had to work around complicated technical limitations in hardware/software integration, AI optimization, and web dashboard. This section explains the critical issues faced with the implementation and the strategic solutions that guaranteed the integrity of the project.

10.1 Optimization of AI Model and Data

10.1.1 Computation Limitations of Edge Hardware:

- **Challenge:** The initial design of the Halm system was aimed at implementing several computer-vision models that would be used to tackle different safety categories simultaneously. Verification showed that Raspberry Pi 5 could not bear the computational load of multiple parallel models without having to significantly decrease the frame rate and increase the load. This was a critical failure point since a fast response time is one of Halm's main non-functional requirements.
- **Resolution:** We chose a common multi-class detection architecture. By uniting the localization of sharp tools, pits, and handheld sharp tools into one YOLOv8 model, we significantly reduced the amount of CPU usage and memory consumption. This integration enabled the edge device to handle all hazards within one inference pass, thus sustaining the real-time performance needed by construction environments.

10.1.2 Fragmentation of Data that does not exist in the real world:

- **Challenge**: The lack of a single good dataset, which contains all of the construction hazards that are relevant to this project, was a major challenge. In addition to this, there was a severe class imbalance in the data that was collected, with "Sharp Tools" overrepresented significantly compared to the other classes.
- **Resolution**: We have used Roboflow to combine three dissimilar data sets into a personalized single repository. In order to reduce the skew, we used data augmentation, such as rotation, motion blur, and variations in brightness, on the minority classes.

10.1.3 Localization inaccuracy in the detection of Sharp Tools:

- **Challenge**: The original performance of sharp tool training was not ideal, often not being able to locate smaller objects on complex backgrounds.
- Resolution: We introduced a high preprocessing pipeline to our code base, including a Garbage Filter that we used to filter out non-normalized coordinates and a converting process that ensured all bounding boxes were normalized based on the actual resolution of the Raspberry Pi Camera V2.

10.1.4 Training Latency and Resource Management:

- **Challenge**: The database grew by merging datasets and the augmentation process, making it prohibitive in terms of training time on standard CPUs, slowing our Agile sprint cycles.
- **Resolution**: We changed our training platform to GPU acceleration. With the A100/T4 infrastructure, we would be able to train the Nano and models within hours instead of days and conduct more frequent tests on the iterative process and optimize the hyperparameter.

10.2 Hardware Integration/Firmware Problems

10.2.1 Critical Hardware Failure (Manufacturing Defect):

- **Challenge**: In the first stage of the hardware assembly, a Raspberry Pi 5 unit was stuck on booting up. The green LED was flashing in an 8-blink sequence. This was determined through research as a manufacturing defect (SDRAM failure) [27].
- **Resolution**: To ensure that we continue following our schedule, we started an immediate process of returning and replacement. Although this brought the hardware testing to a temporary halt, we shifted the attention to dashboard development and AI training to make sure we were on track with Milestone 3 goals.

10.2.2 Power Supply and Amperage Stabilization:

- **Challenge**: To support heavy AI workloads, the Raspberry Pi 5 has a very high stability in the power source (5V/5A (25W)). The standard portable power banks cannot maintain this amperage, and the system does not boot.
- **Resolution**: We replaced the power bank with the official Raspberry Pi USB-C power supply. This guaranteed that the board, active cooler, and other sensors had a steady power supply, hence eliminating loss of data and maintaining the high-speed inference property of the system.

10.2.3 CSI Port Compatibility:

- **Challenge**: The physical configuration of the Raspberry Pi 5 utilizes a smaller 22-pin CSI port for the camera. We found that 22-to-22 pin cables were not readily available in the local market, which threatened to delay the integration of the Smart Vision module.
- **Resolution**: We changed our hardware choice to include a camera module that can fit a 15-pin port. This allowed a specialized 15-to-22-pin Flexible Flat Cable (FFC) to be used, which successfully interposed the connection between the camera and the board without reducing the bandwidth of the video.

10.2.4 Firmware and Command Line Driver Errors:

- **Challenge**: Within the software-to-hardware integration, a range of Linux-level errors were encountered, such as “libcamera-hello: command not found” and failures in the Picamera2 library. These bugs were due to unsuitable versions of OS and the unavailable hardware interfaces in the base version.
- **Resolution**: We did a complete update of the Raspberry Pi and used “raspi-config” utility to configure the camera interfaces manually. Afterwards, we installed the required Python dependencies of the Ultralytics and OpenCV engines, thus creating a consistent method of communication between the hardware and our YOLOv8 model.

10.3 Web Dashboard & Full-Stack Integration

10.3.1 UI/UX Design and Elements Hierarchy:

- **Challenge**: The main challenge was to create a professional interface that was still easy to use and intuitive at the same time, but that would render complex safety data. Specifically, the Safety Live Feed component (AlertFeed.tsx) was growing out of control when the artificial intelligence subsystem identified a range of hazards at the same time, leading to overlaps between other user-interface items and thus disrupting the visual balance.
- **Resolution**: The alert element was restructured into a scrollable and logically arranged one. A sorting algorithm was adopted as a Newest-on-Top.

10.3.2 Database Schema Evolution for Emergency Response:

- **Challenge**: Halfway through the development process it was realized that the current database structure could not support the safety officers and paramedics in the event of any medical emergencies. Essential information on health was not included in the first scheme, thus crippling the effectiveness of the system in providing quick intervention.
- **Resolution**: The SQLite scheme was edited to add a separate table, which has such attributes as blood type and chronic medical conditions. Complete CRUD (Create, Read, Update, Delete) was supported through "WorkersPage.tsx" so that safety officers could be able to maintain correct and up-to-date profile data.

10.3.3 Transition from Static to Dynamic Data (Flask Integration):

- **Challenge**: When in the prototype phase, the top-level KPI cards (e.g., Total Workers, Active Hazards, Resolved Incidents) showed a fixed or a mock value in the HTML. As a result, the real-time changes in numerical values were not observed upon the identification of a new hazard or the addition of a worker, which made the dashboard useless in live monitoring.
- **Resolution**: The front end was refactored to get real-time values on the Flask backend. The use of static placeholders was substituted with dynamically

generated API calls, which makes the KPI cards and other tables under the cards update automatically when a new hazard is registered with the AI engine (best.pt) or each of the status changes in the database.

11 Work timeline

The project timeline is structured across 15 weeks, starting with the Preparing and Research phase to establish the background and objectives. This is followed by the Literature Review (Weeks 3–7) to identify research gaps, alongside the Introduction and Scope phases (Weeks 4–7) to define requirements and deliverables. The core technical approach is then planned in the Methodology phase (Weeks 6–8), leading into the Report Writing phase (Weeks 7–9) and the submission of Milestone 1 in Week 9. Following this, the Design and Analysis phase (Weeks 12–13) focuses on developing the system architecture, interaction flow, database design, flowchart, use case, and hardware connection diagrams. In parallel, the Technical Development Plan (Weeks 12–14) defines the selected tools, technologies, and implementation timeline, while the final report and presentation are completed, concluding with the submission of Milestone 2 in Week 15.

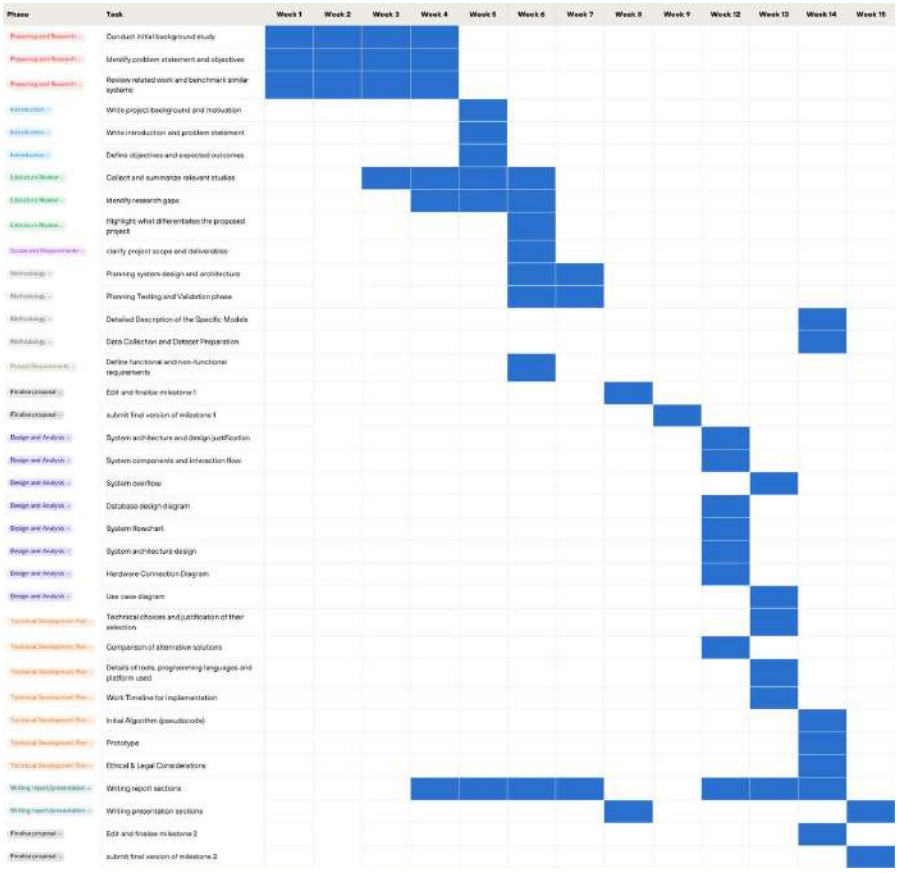

Figure 23: Work Timeline.

12 Work Timeline for Implementation

The implementation is structured using seven accelerated two-week sprints, prioritizing data preparation and model training (O1) to drive the subsequent hardware integration and integration phases. This schedule maintains the necessary logical progression of an Agile project, ensuring continuous testing and validation of the system's core safety pillars (O1, O2, O3).

ID	Task Name	Start	End	Duration
1	Phase I: Vision & Connectivity (O1)	2026-01-06	2026-02-04	28 days
2	P1.1: Dataset Curation & Labeling + RPi Setup	2026-01-06	2026-01-21	14 days
3	P1.2: CV Model Training & T-SIM/4G/GPS In...	2026-01-22	2026-02-04	14 days
4	Phase II: Hardware & Kinematics (O2)	2026-02-05	2026-03-04	28 days
5	P2.1: Final Hardware Integration	2026-02-05	2026-02-18	14 days
6	P2.2: Kinematic Algorithm Dev & Unit Testing	2026-02-19	2026-03-04	14 days
7	Phase III: Dashboard & Data Fusion (O3)	2026-03-05	2026-04-01	28 days
8	P3.1: Dashboard Front-end Dev & Cloud Ba...	2026-03-05	2026-03-18	14 days
9	P3.2: Database Integration & Multi-modal Da...	2026-03-19	2026-04-01	14 days
10	Phase IV: Final Validation & Handover	2026-04-05	2026-04-18	14 days
11	P4.1: Stress Test, Final Tuning & Document...	2026-04-05	2026-04-18	14 days

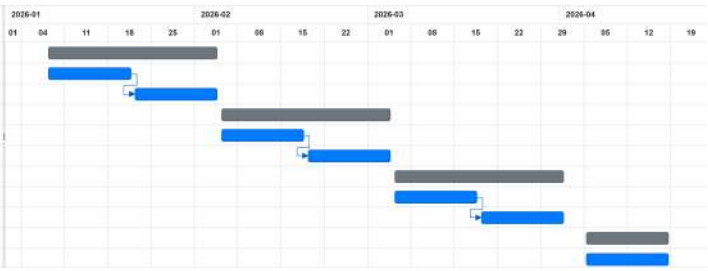

Figure 24: Work Timeline for Implementation.

References

[1] Saudi Vision 2030. (n.d.). *Overview*. Retrieved September 27, 2025, from https://www.vision2030.gov.sa/en/overview

[2] Jawaharlalnehru, M., Begum, Y. A., Ajeeth, A., Nitheesh, N., Krishnan, N. Y., & Murali, A. (2025). IoT-based Smart Helmet and Jacket for Real-Time Safety Monitoring in Construction Site. In *Proceedings of the 7th International Conference on Inventive Material Science and Applications (ICIMA-2025)* (pp. 1070–1077). IEEE.

[3] Sivadasan, J., Raj, R. A., Kumar, M. S., Sarathkumar, D., Hemanth, A., & Sailaesh, A. (2025). Safety Monitoring for Construction Workers Using RSSI. In *2025 International Conference on Electronics, AI and Computing (EAIC)* (pp. 1–6). IEEE.

[4] Ahmed, M., El-Neaman, C., Al-Korki, H., El-Abd, M., Al-Hazza, D., & Esmaeili, S. (2024). The Smart Helmet System: Towards Providing Safer Construction Work Sites. In *2024 IEEE Middle East Conference on Communications and Networking (MECOM)* (pp. 1–5). IEEE.

[5] General Organization for Social Insurance. (2024, March 25). *GOSI: Work injuries decreased by 8.5% during 2023*. Saudi Press Agency.

[6] Vinay, R., Reddy, C. M., Reddy, R. T., Fernandez, K. A., Brunda, V., & D'Costa, J. R. (2025). Enviro-Alert Helmet: A Wearable Internet of Things Solution for Safety, Hazard Detection, and Rapid Response. In *Proceedings of the Third International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT-2025)* (pp. 1065–1069). IEEE.

[7] Vaishnavi, T., Kumaravel, G., Vijaya, G., Archana, T., Rahul, D., & Kirthiga, S. (2025). SAFE-Net: Secure AI-driven Forecasting and Efficiency Network for Smart Helmet Systems. In *2025 International Conference on Recent Advances in Electrical, Electronics, Ubiquitous Communication, and Computational Intelligence (RAEEUCCI)* (pp. 1–6). IEEE.

[8] Jayasree, V., & Nivetha Kumari, M. (2020). IoT based Smart Helmet for Construction Workers. In *2020 IEEE 7th International Conference on Smart Structures and Systems (ICSSS)* (pp. 1–6). IEEE.

[9] King Faisal University. (n.d.). *Vision, mission and goals*. King Faisal University. https://www.kfu.edu.sa/en/Deans/Research/Pages/VandM.aspx

[10] Jeong, M., Lee, H., Bae, M., Shin, D.-B., Lim, S.-H., & Lee, K. B. (2018). Development and application of the smart helmet for disaster and safety. In *2018 International Conference on Information and Communication Technology Convergence (ICTC)* (pp. 1084–1089). IEEE.

[11] Chen, D., Zhang, Z., Liu, Y., & Yang, X. T. (2025, February). INSIGHT: Enhancing autonomous driving safety through vision-language models on context-aware hazard detection and edge case evaluation (arXiv:2502.00262) [Preprint]. *ArXiv*.

[12] Radhika, M., Kumar G, R., S., Gopinathan, S., V, H., Kumar R, P., & Balaji Elumalai, R. (2024). IoT-enabled safety helmet for coal miners: Real-time monitoring, data collection, and analysis. *Proceedings of the 2024 International Conference on Smart Technologies for Sustainable Development Goals (ICSTSDG)*, 979–984.

[13] King Faisal University. (2025). King Faisal University Identity. In KFU Strategic Documents Series (pp. 4–40). King Faisal University.

[14] Lee, Y. et al. (2022). Trends in Smart Safety with Multimodal Sensing for Health and Safety: Scoping Review.

[15] Choi, Y., & Kim, Y. (2021). Applications of Smart Helmets in Applied Sciences: A Systematic Review.

[16] Kumar, P. K. R. et al. (2023). Smart Helmet based Accident Detection and Notification System for Two-Wheeler Motorcycles.

[17] Navasree, V. et al. (2020). A Survey on Smart Safety Helmets and Their Role in Disaster Management.

[18] Suresh, K. (2022). 'Smart Construction Safety Helmet': A Construction Safety Tool Embedded with Health Monitoring and Salary Deduction Function.

[19] Battistoni, C. (2021). An IoT-Based Mobile System for Safety Monitoring of Lone Workers.

[20] Mila, S. P., & Roy, J. (2023). Design a Smart Helmet as a Quick Responder Safety Solution for the Bikers.

[21] Kalita, J. (2023). Smart Riding with IoT Helmet: A Step Towards Road Safety.

[22] Nnaji, C. et al. (2021). Wearable Sensing Devices: Towards the Development of a Personalized System for Construction Safety and Health Risk Mitigation.

[23] DatasetNinja, "Small Size Construction Tools Dataset," 2022. [Online]. Available: https://datasetninja.com/small-size-construction-tools

[24] Chitholian, "Annotated Potholes Dataset," Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/chitholian/annotated-potholes-dataset

[25] Zoya77, "AGRO Human Activity IMU Sensor Dataset," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/zoya77/agro-human-activity-imu-sensor-dataset

[26] Saudi Arabia, Personal Data Protection Law – English Version, Apr. 2023.

[27] Raspberry Pi Ltd., "LED warning flash codes," Raspberry Pi Documentation. [Online]. Available: https://www.raspberrypi.com/documentation/computers/configuration.html#led-warning-flash-codes

[28] Roboflow, "High-Resolution Potholes Dataset," 2023. [Online]. Available: https://universe.roboflow.com/rana-7krlb/pits-wqyxv-h510m/browse?queryText=&pageSize=50&startingIndex=0&browseQuery=true

[29] J. Cmmsh, "Hand Tool Detection Dataset," Roboflow Universe, 2023. [Online]. Available: https://universe.roboflow.com/jee-cmmsh/hand-tool-detection-pbu9x/browse